Introduction to processes

Early computer system allowed any one program to be executed at a once, current days computer system allow multiple programs to be loaded in memory and to be executed at the same time or step by step. This executing multiple programs is being resulted as process. A process is the unit of work in a modern time-sharing system.

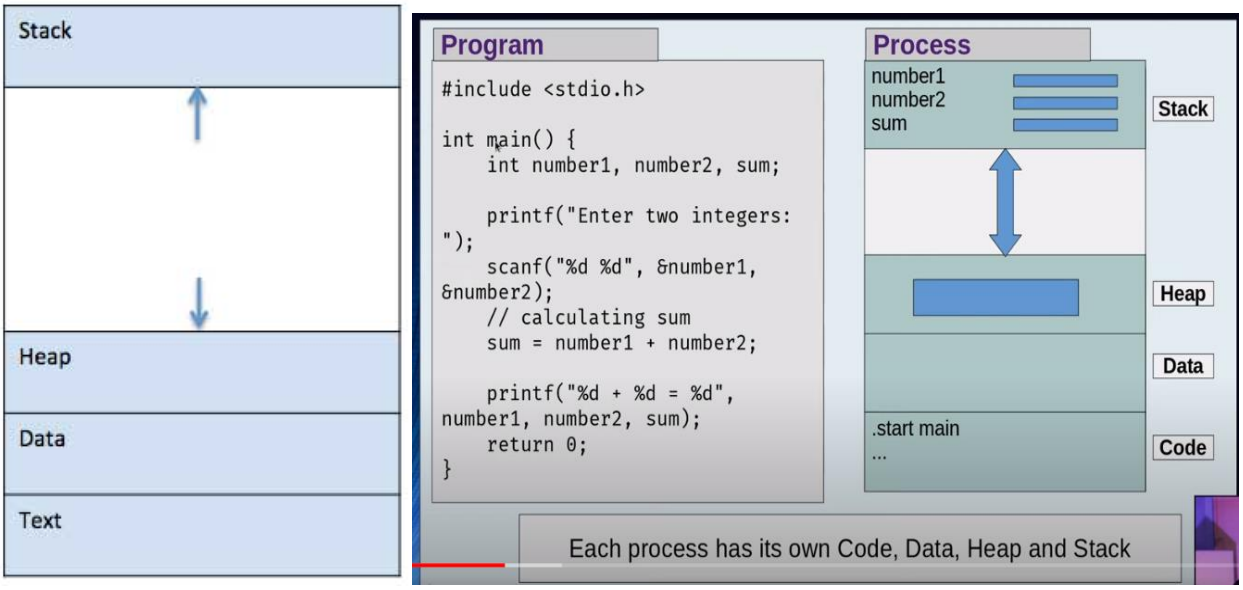
What is a Process?

A process is a program in execution. Process is not as same as program code but a lot more than it. A process is an 'active' entity as opposed to program which is considered to be a 'passive' entity. Attributes held by process include hardware state, memory, CPU etc.

When a program is loaded into the memory and it becomes a process, it can be divided into four sections ─ **stack, heap, text and data**. The following image shows a simplified layout of a process inside main memory.

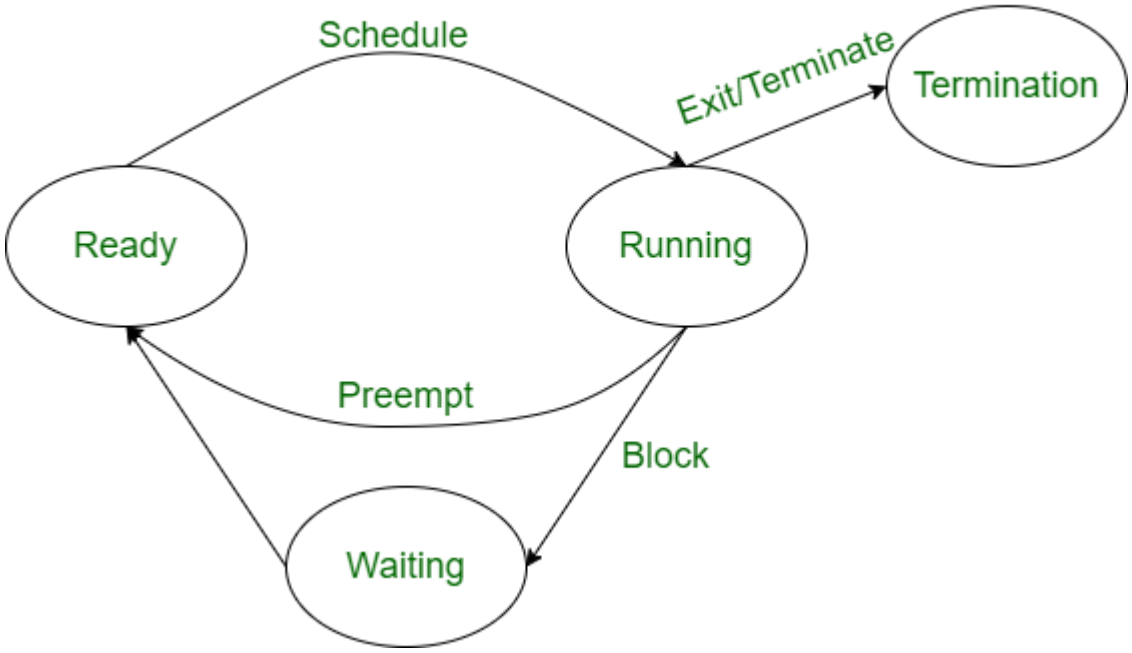
Process memory is divided into four sections for efficient working:

- The **Text section** is made up of **the compiled program code, read in from non-volatile storage** when the program is launched.
- The **Data section** is made up of the **global and static variables, allocated and initialized prior to executing the main.**
- The **Heap** is used for the dynamic memory allocation, and is managed via calls to new, delete, malloc, free, etc.
- The **Stack** is used for local variables. Space on the stack is reserved for **local variables when they are declared**.



Operation on a Process:

The execution of a process is a complex activity. It involves various operations. Following are the operations that are performed while execution of a process:



1. Creation: This the initial step of process execution activity. Process creation means the construction of a new process for the execution. This might be performed by system, user or old process itself. There are several events that leads to the process creation. Some of the such events are following:

- When we start the computer, system creates several background processes.
- A user may request to create a new process.
- A process can create a new process itself while executing.

2. Scheduling/Dispatching: The event or activity in which the state of the process is changed from ready to running. It means the operating system puts the process from ready state into the running state. Dispatching is done by operating system when the resources are free or the process has higher priority than the ongoing process. There are various other cases in which the process in running state is preempted and process in ready state is dispatched by the operating system.

3. Blocking: When a process invokes an input-output system call that blocks the process and operating system put in block mode. Block mode is basically a mode where process waits for input-output. Hence, on the demand of process itself, operating system blocks the process and dispatches another process to the processor. Hence, in process blocking operation, the operating system puts the process in 'waiting' state.

4. Preemption: When a timeout occurs that means the process hadn't been terminated in the allotted time interval and next process is ready to execute, then the operating system preempts the process. This operation is only valid where CPU scheduling supports preemption. Basically, this happens in priority scheduling where on the incoming of high priority process the ongoing process is preempted. Hence, in process preemption operation, the operating system puts the process in 'ready' state.

5. Termination: Process termination is the activity of ending the process. In other words, process termination is the relaxation of computer resources taken by the process for the execution. Like creation, in termination also there may be several events that may lead to the process termination. Some of them are:

- Process completes its execution fully and it indicates to the OS that it has finished.
- Operating system itself terminates the process due to service errors.
- There may be problem in hardware that terminates the process.

- One process can be terminated by another process.

States of Process:

A process is in one of the following states:

1. New: Newly Created Process (or) being-created process. A program which is going to be picked up by the OS into the main memory is called a new process.
2. **Ready:** After creation process moves to Ready state, i.e. the process is ready for execution. Whenever a process is created, it directly enters in the ready state, in which, it waits for the CPU to be assigned. The OS picks the new processes from the secondary memory and put all of them in the main memory.

The **processes which are ready for the execution and reside in the main memory are called ready state processes**. There can be many processes present in the ready state.

3. Run: Currently running process in CPU (only one process at a time can be under execution in a single processor). One of the processes from the ready state will be chosen by the OS depending upon the scheduling algorithm. Hence, if we have only one CPU in our system, the number of running processes for a particular time will always be one. If we have n processors in the system then we can have n processes running simultaneously.

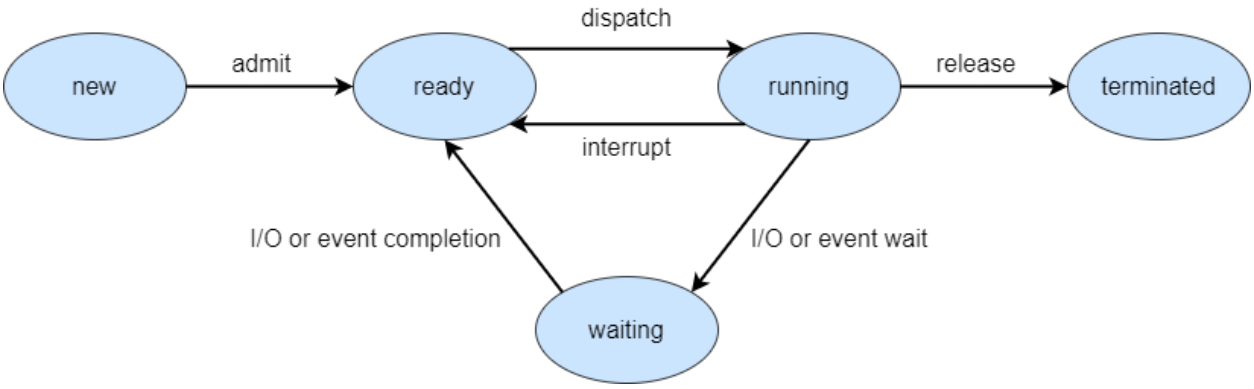
4. Wait (or Block): When a process requests I/O access. When a process waits for a certain resource to be assigned or for the input from the user then the OS move this process to the block or wait state and assigns the CPU to the other processes.

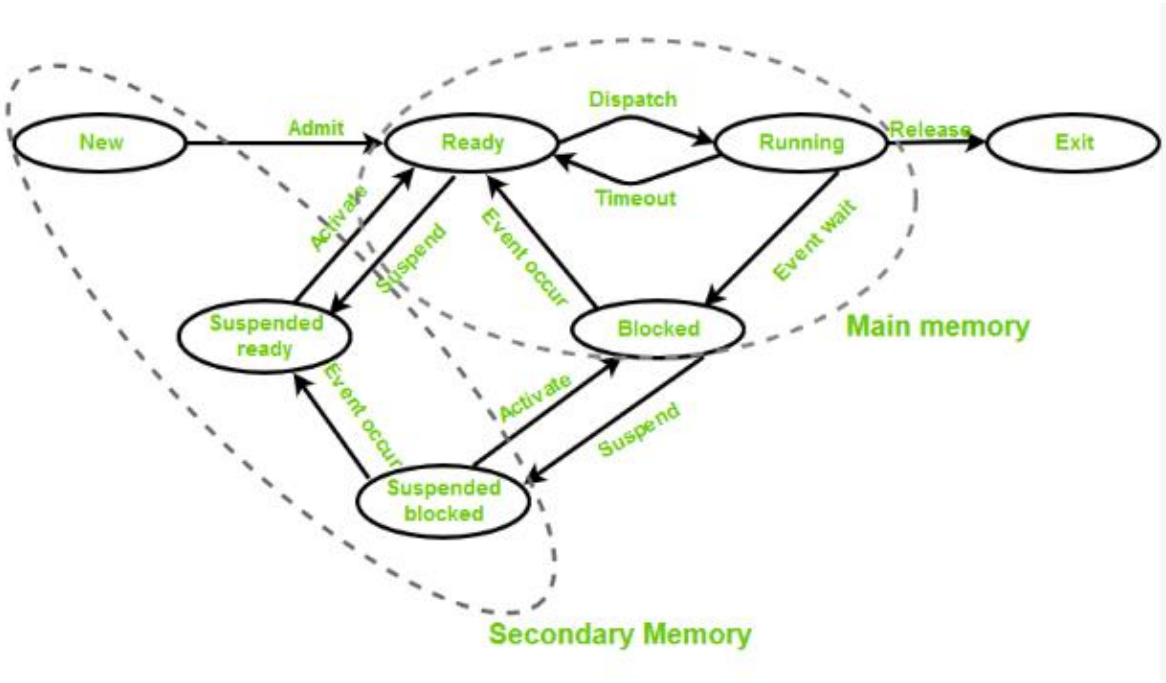
5. Complete (or terminated): The process completed its execution. When a process finishes its execution, it comes in the termination state. All the context of the process (Process Control Block) will also be deleted the process will be terminated by the Operating system.

6. Suspended Ready: When the ready queue becomes full, some processes are moved to suspended ready state. A process in the ready state, which is moved to secondary memory from the main memory due to lack of the resources (mainly primary memory) is called in the suspend ready state.

If the main memory is full and a higher priority process comes for the execution then the OS have to make the room for the process in the main memory by throwing the lower priority process out into the secondary memory. The suspend ready processes remain in the secondary memory until the main memory gets available.

7. Suspended Block: When waiting queue becomes full. Instead of removing the process from the ready queue, it's better to remove the blocked process which is waiting for some resources in the main memory. Since it is already waiting for some resource to get available hence it is better if it waits in the secondary memory and make room for the higher priority process. These processes complete their execution once the main memory gets available and their wait is finished.





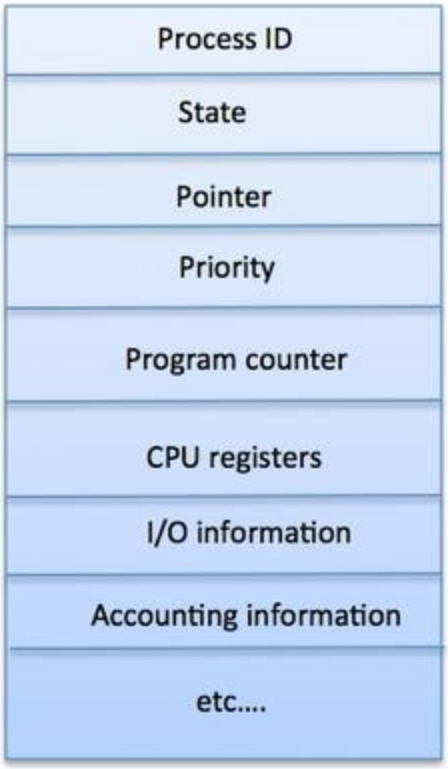
Process Control Block (PCB)

A Process Control Block is a data structure maintained by the Operating System for every process. The PCB is identified by an integer process ID (PID). A PCB keeps all the information needed to keep track of a process. There is a Process Control Block for each process, enclosing all the information about the process. It is a data structure, which contains the following:

S.N.	Information & Description
1	**Process State** The current state of the process i.e., whether it is ready, running, waiting, or whatever.
2	**Process privileges** This is required to allow/disallow access to system resources.
3	**Process ID** Unique identification for each of the process in the operating system.
4	**Pointer** A pointer to parent process.
5	**Program Counter** Program Counter is a pointer to the address of the next instruction to be executed for this process.
6	**CPU registers** Various CPU registers where process need to be stored for execution for running state.

7	**CPU Scheduling Information** Process priority and other scheduling information which is required to schedule the process.
8	**Memory management information** This includes the information of page table, memory limits, Segment table depending on memory used by the operating system.
9	**Accounting information** This includes the amount of CPU used for process execution, time limits, etc.
10	**IO status information** This includes a list of I/O devices allocated to the process.

The architecture of a PCB is completely dependent on Operating System and may contain different information in different operating systems. Here is a simplified diagram of a PCB −



The PCB is maintained for a process throughout its lifetime, and is deleted once the process terminates.

Deadlock

Every process needs some resources to complete its execution. However, the resource is granted in a sequential order.

1. The process requests for some resource.
2. OS grant the resource if it is available otherwise let the process waits.
3. The process uses it and release on the completion.

Deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.
Consider an example when two trains are coming toward each other on same track and there is only one track, none of the trains can move once they are in front of each other. Similar situation occurs in operating systems when there are two or more processes hold some resources and wait for resources held by other(s). For example, in the below diagram, Process 1 is holding Resource 1 and waiting for resource 2 which is acquired by process 2, and process 2 is waiting for resource 1.

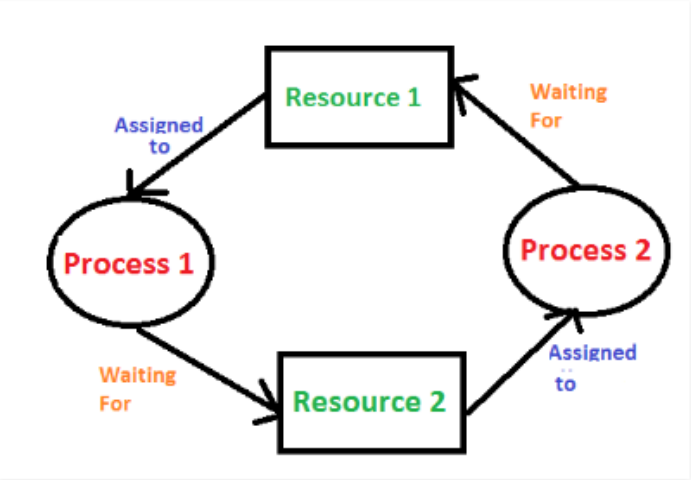


Fig: Deadlock

Necessary conditions for Deadlocks

1. **Mutual Exclusion**

   A resource can only be shared in mutually exclusive manner. It implies, if two process cannot use the same resource at the same time.

2. **Hold and Wait**

   A process waits for some resources while holding another resource at the same time.

3. **No preemption**

   The process which once scheduled will be executed till the completion. No other process can be scheduled by the scheduler meanwhile.

4. **Circular Wait**

   All the processes must be waiting for the resources in a cyclic manner so that the last process is waiting for the resource which is being held by the first process.

Preemptable and Non-preemptable Resources

The **resource** is the object granted to a process.

Preemptable and Non-preemptable Resources

- Resources come in two types
  1. **Preemptable**, meaning that the resource can be taken away from its current owner (and given back later). An example is memory.
  2. **Non-preemptable**, meaning that the resource cannot be taken away. An example is a printer, CD.
- Life history of a resource is a sequence of
  1. Request
  2. Allocate
  3. Use
  4. Release
- Processes make requests, use the resource, and release the resource. The allocate decisions are made by the system and there are various policies used to make these decisions.

Deadlock modeling

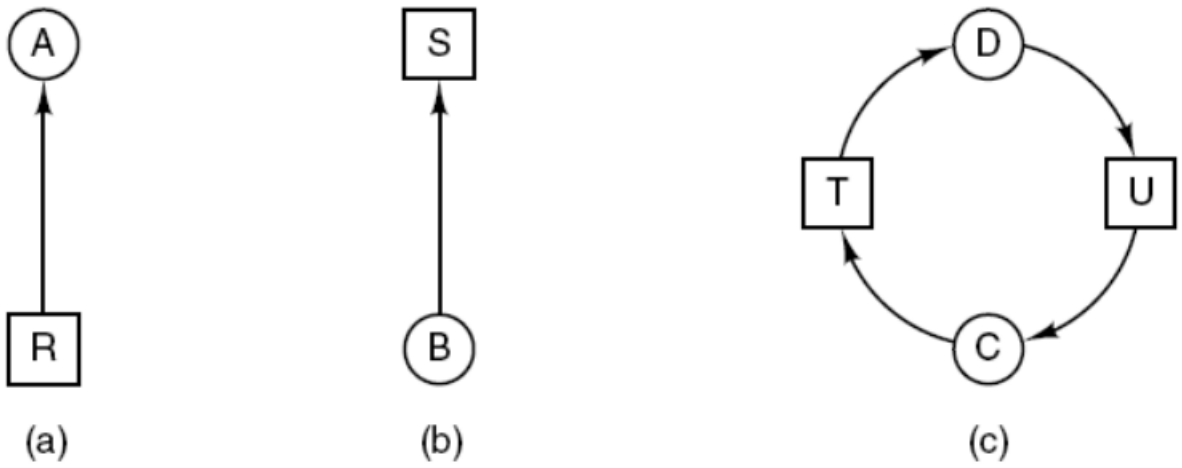
These four conditions can be modeled using **resource allocation graph** is a directed graph that is used for describing and reasoning about deadlock.

The graph's set of vertices V consists of two partitions: P = {P1, P2, P3, …, Pn} is the set of active processes, while R = {R1, R2, R3, …, Rm} is the set of available resource types.

The two kinds of vertices (nodes): **processes, shown as circles, and resources, shown as squares**.

The graph's set of edges E is also partitioned into two types: request edges (Pi→Rj) indicate that Pi is waiting for an instance of Rj, while assignment edges (Rj→Pi) indicate that Rj instance has been allocated to Pi.

An arc from a resource node (square) to a process node (circle) means that the resource has previously been requested by, granted to, and is currently held by that process. In fig, resource *R* is currently assigned to process *A* and Process B is requesting for resource S. An arc from a process node (circle) to a resource node (square) means that the process is currently blocked/waiting for that resource.



(a) (b) (c)

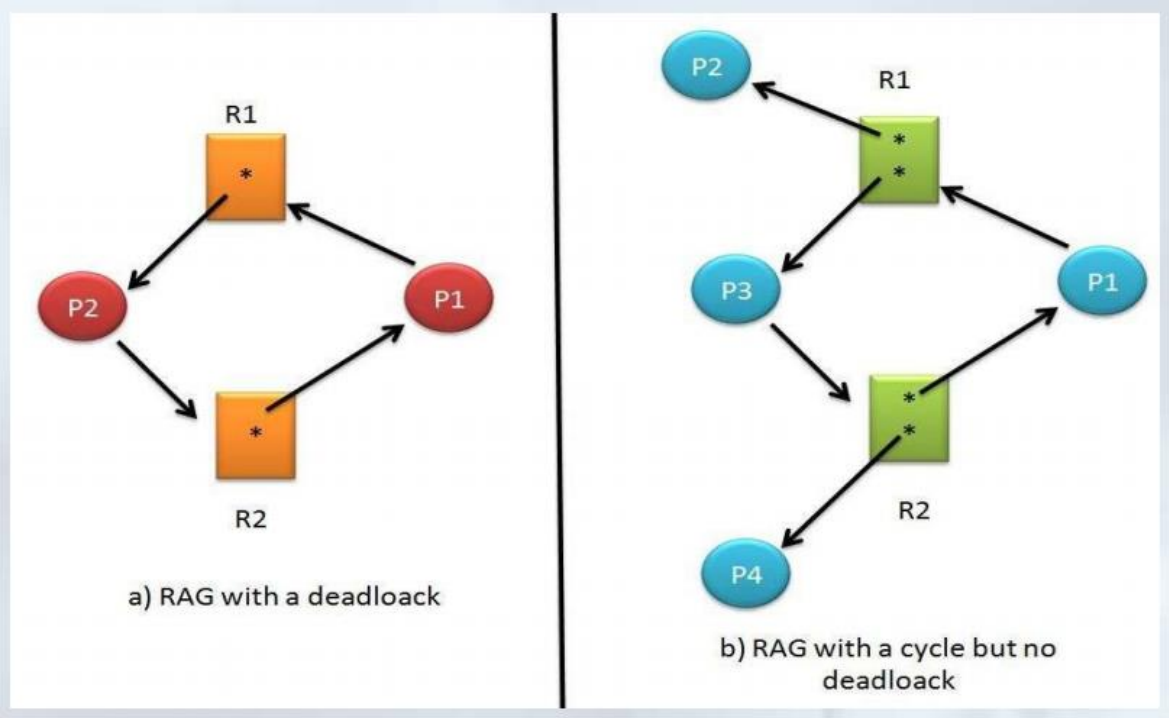
Fig:Resource allocation graph

By expressing some combination of processes, resource types, requests, and allocations as a resource-allocation graph, certain conclusions can be drawn based on the graph's structure:

• If the graph has no cycles, then there is no deadlock.

• If the graph does have a cycle, then there might be deadlock (i.e., the existence of a cycle is necessary but not sufficient):

- If the resource types involved in the cycle have only one instance(only one instance of that resource), then we have deadlock.In other words, the existence of a cyclebecomes necessary and sufficient when we only have one instance per resource type in the cycle.
- If the resource types involved in the cycle have more than one instance (multiple instances of same resource), then there may and may not have deadlock.



There are 4 ways to overcome deadlock

- Deadlock avoidance
- Deadlock prevention
- Deadlock detection and recovery
- Deadlock ignorance

Deadlock Avoidance (Deadlock detection)

Banker's Algorithm

Banker's Algorithm is resource allocation and deadlock avoidance algorithm which test all the request made by processes for resources, it runs safety algorithm checks for the safe state, if after granting request system remains in the safe state it allows the request and if there is no

safe state it doesn't allow the request made by the process. Banker's algorithm ensures that the system will never enter in an unsafe state.

Inputs to Banker's Algorithm:

1. Max need of resources by each process.
2. Currently allocated resources by each process.
3. Max free available resources in the system.

The request will only be granted under the below condition:

1. If the request made by the process is less than or equal to max need to that process.
2. If the request made by the process is less than or equal to the freely available resource in the system.

Solution

Given resources A=10 B=5 C=7

Process	Allocation	Max	Available
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Question1. What will be the content of the Need matrix?

Need [i, j] = Max [i, j] – Allocation [i, j]

So, the content of Need Matrix is:

Process	Need		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

Deadlock Prevention

We can prevent Deadlock by eliminating any of the above four conditions.

Eliminate Mutual Exclusion

It is not possible to dis-satisfy the mutual exclusion because some resources, such as the tape drive and printer, are inherently non-sharable.

Eliminate Hold and wait

1. Allocate all required resources to the process before the start of its execution, this way hold and wait condition is eliminated but it will lead to low device utilization. for example, if a process requires printer at a later time and we have allocated printer before the start of its execution printer will remain blocked till it has completed its execution.

Eliminate No Preemption

The no-preemption condition can be alleviated by forcing a process waiting for a resource that cannot immediately be allocated to relinquish all of its currently held resources, so that other processes may use them to finish. This strategy requires that when a process that is holding some resources is denied a request for additional resources. The process must release its held resources and, if necessary, request them again together with additional resources. Implementation of this strategy denies the "non-preemptive" condition effectively.

Eliminate Circular Wait

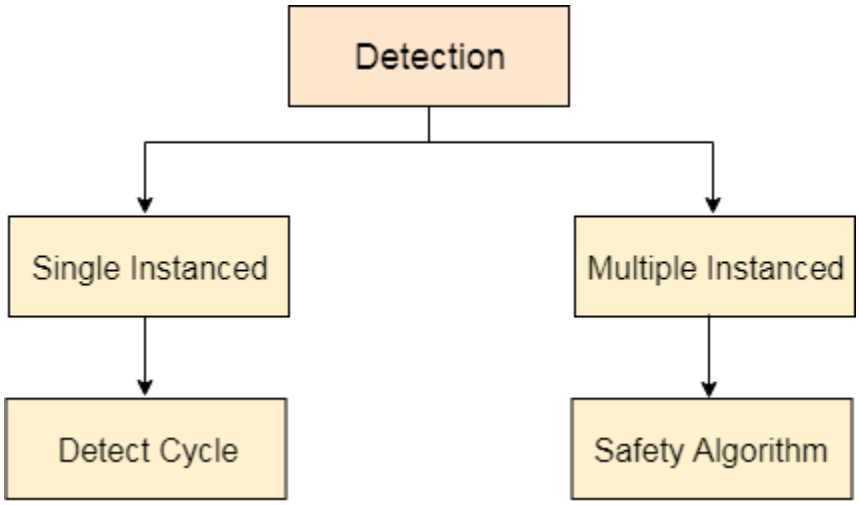
Each resource will be assigned with a numerical number. A process can request the resources increasing/decreasing order of numbering.
For Example, if P1 process is allocated R5 resources, now next time if P1 ask for R4, R3 lesser than R5 such request will not be granted, only request for resources more than R5 will be granted. The problem with this strategy is that it may be impossible to find an ordering that satisfies everyone.

Deadlock Detection and recovery

In this approach, The OS doesn't apply any mechanism to avoid or prevent the deadlocks. Therefore, the system considers that the deadlock will definitely occur. In order to get rid of deadlocks, The OS

periodically checks the system for any deadlock. In case, it finds any of the deadlock then the OS will recover the system using some recovery techniques.

The OS can detect the deadlocks with the help of Resource allocation graph.

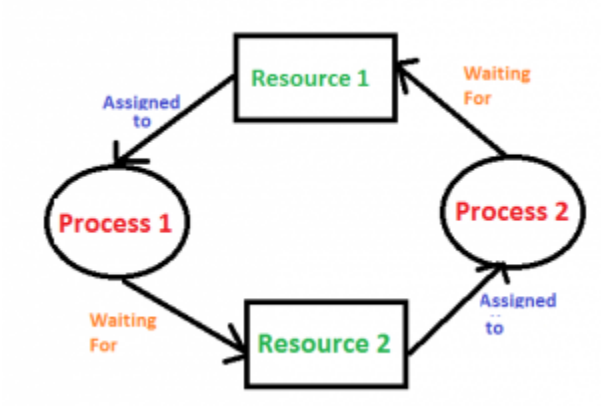


In single instanced resource types, if a cycle is being formed in the system, then there will definitely be a deadlock. On the other hand, in multiple instanced resource type graph, detecting a cycle is not just enough.We have to apply the safety algorithm on the system by converting the resource allocation graph into the allocation matrix and request matrix.

Deadlock Detection

1. **If resources have single instance:**

   In this case, for Deadlock detection we can run an algorithm to check for cycle in the **Resource Allocation Graph**. Presence of cycle in the graph is the sufficient condition for deadlock.



   In the above diagram, resource 1 and resource 2 have single instances. There is a cycle R1 → P1 → R2 → P2. So, Deadlock is Confirmed.

2. **If there are multiple instances of resources:**

   Detection of the cycle is necessary but not sufficient condition for deadlock detection. In this case, the system may or may not be in deadlock varies according to different situations.

   In a Resource Allocation Graph where all the resources are NOT single instance,

   - If a cycle is being formed, then system may be in a deadlock state.

- Banker's Algorithm is applied to confirm whether system is in a deadlock state or not.
- If no cycle is being formed, then system is not in a deadlock state. Presence of a cycle is a necessary but not a sufficient condition for the occurrence of deadlock.

Deadlock recovery

When a Deadlock Detection Algorithm determines that a deadlock has occurred in the system, the system must recover from that deadlock.

There are two approaches of breaking a Deadlock:

1. Process Termination:

To eliminate the deadlock, we can simply kill one or more processes.

For this, we use two methods:

(a)Abort all the Deadlocked Processes:

Aborting all the processes will certainly break the deadlock, but with a great expense. The deadlocked processes may have computed for a long time and the result of those partial computations must be discarded and there is a probability to recalculate them later.

(b) Abort one process at a time until deadlock is eliminated:

Abort one deadlocked process at a time, until deadlock cycle is eliminated from the system. Due to this method, there may be considerable overhead, because after aborting each process, we have to run deadlock detection algorithm to check whether any processes are still deadlocked.

2. Resource Preemption:

To eliminate deadlocks using resource preemption, we preempt some resources from processes and give those resources to other processes. This method will raise three issues –

(a) Selecting a victim:

We must determine which resources and which processes are to be preempted in order to minimize the cost.

(b) Rollback:

We must determine what should be done with the process from which resources are preempted. One simple idea is total rollback. That means abort the process and restart it.

(c) Starvation:

In a system, it may happen that same process is always picked as a victim. As a result, that process will never complete its designated task. This situation is called **Starvation** and must be avoided. One solution is that a process must be picked as a victim only a finite number of times.

Deadlock Ignorance (Ostrich Algorithm)

- Easiest way to deal with the problem
- Pretend that there is no problem
- This strategy says that stick your head into sand and pretend that there is no problem at all.

- This strategy suggests to ignore the deadlock because deadlock occur very rare, but the system crashes due to the hardware failure, compiler error, and operating system bugs frequently, then not to pay large penalty in the system performance or convince to eliminate deadlocks.

Issues related to deadlock

There is real danger of deadlock whenever multiple processes are running at the same instance of time.

The approach used is called two phase locking

In first phase, the process tries to lock all the records that it needs, one at a time. And if it succeeds, then it begins the second phase, performing its updates and releasing the locks. No any real work is done in the first phase.

If during the first phase, some records that is already locked, the process just releases all its locks and starts the first phase all over.

It is very difficult to implement in real situations.

Non-resource deadlocks:

Deadlock occurring without acquiring any resources by any processes. It is the situation when each process is waiting other process to do something.

For example: say you have two nodes in a network that communicate with each other and the following scenario occurs:

- Node1 sends a message to node 2 and waits for a response.
- Node2 receives the message and sends back the response to node1 and waits.
- But the response is lost on the network due to a temporary disruption.

  Both nodes are waiting for each other => Deadlock

Starvation

Starvation or indefinite blocking is phenomenon associated with the Priority scheduling algorithms, in which a process ready to run for CPU can wait indefinitely because of low priority. In heavily loaded computer system, a steady stream of higher-priority processes can prevent a low-priority process from ever getting the CPU.

Some of the common causes of starvation are as follows:

1. If a process is never provided the resources it requires for execution because of faulty resource allocation decisions, then starvation can occur.
2. A lower priority process may wait forever if higher priority processes constantly monopolize the processor.

3. Starvation may occur if there are not enough resources to provide to every process as required.
4. If random selection of processes is used then a process may wait for a long time because of non-selection.

Some solutions that can be implemented in a system to handle starvation are as follows:

1. An independent manager can be used for allocation of resources. This resource manager distributes resources fairly and tries to avoid starvation.
2. Random selection of processes for resource allocation or processor allocation should be avoided as they encourage starvation.
3. The priority scheme of resource allocation should include concepts such as **aging,** where the priority of a process is increased the longer it waits. This avoids starvation.

There has been rumors that in 1967 Priority Scheduling was used in IBM 7094 at MIT, and they found a low-priority process that had not been submitted till 1973.

Assignment2: What is aging? Differentiate between deadlock and starvation.

Chapter 2 - Scheduling

The process scheduling is the activity of the process manager that handles the removal of the running process from the CPU and the selection of another process on the basis of a particular strategy. Process scheduling is an essential part of a Multiprogramming operating systems. Such operating systems allow more than one process to be loaded into the executable memory at a time and the loaded process shares the CPU using time multiplexing.

Objective and criteria of process scheduling

- Max CPU utilization [Keep CPU as busy as possible]
- Fair allocation of CPU.
- Max throughput [Number of processes that complete their execution per time unit]
- Min turnaround time [Time taken by a process to finish execution]
- Min waiting time [Time a process waits in ready queue]
- Min response time [Time when a process produces first response]

SchedulingLevel

Schedulers are special system software which handle process scheduling in various ways. Their main task is to select the jobs to be submitted into the system and to decide which process to run. Schedulers are of three levels:

- Long-Term Scheduler
- Short-Term Scheduler
- Medium-Term Scheduler

Long Term Scheduler

It is also called a job scheduler. A long-term scheduler determines which programs are admitted to the system for processing. It selects processes from the queue and loads them into memory for execution. Process loads into the memory for CPU scheduling. It also controls the degree of multiprogramming.

Short Term Scheduler

It is also called as CPU scheduler. Its main objective is to increase system performance in accordance with the chosen set of criteria. It changes ready state to running state of the process. CPU scheduler selects a process among the processes that are ready to execute and allocates CPU to one of them.Short-term schedulersmake the decision of which process to execute next. Short-term schedulers are faster than long-term schedulers.

Medium Term Scheduler

Medium-term scheduling is a part of swapping. It removes the processes from the memory. It reduces the degree of multiprogramming. The medium-term scheduler is in-charge of handling the swapped out-processes.

A running process may become suspended if it makes an I/O request. A suspended process cannot make any progress towards completion. In this condition, to remove the process from memory and make space for other processes, the suspended process is moved to the secondary storage. This process is called swapping, and the process is said to be swapped out or rolled out.

	Long-Term Scheduler	Short-Term Scheduler	Medium-Term Scheduler
1	It is a job scheduler	It is a CPU scheduler	It is a process swapping scheduler.
2	Speed is lesser than short term scheduler	Speed fastest among other two	Speed is in between both short and long term scheduler.
3	It controls the degree of multiprogramming	It provides lesser control over degree of multiprogramming	It reduces the degree of multiprogramming
4	It selects processes from pool and loads them into memory for Execution	It selects those processes which are ready to execute	It can re-introduce the process into memoryand execution can be continued

Preemptive vs Non-PreemptiveScheduling

1. **Preemptive Scheduling:**

Preemptive scheduling is used when a process switches from running state to ready state or from waiting state to ready state. The resources (mainly CPU cycles) are allocated to the process for the limited amount of time and then is taken away, and the process is again placed back in the ready queue if that process still has CPU burst time remaining. That process stays in ready queue till it gets next chance to execute.

Algorithms based on preemptive scheduling are: Round Robin (RR), Shortest Job First (SJF preemptive version) and Priority (preemptive version), etc.

2. **Non-Preemptive Scheduling:**

Non-preemptive Scheduling is used when a process terminates, or a processs which from running to waiting state. In this scheduling, once the resources (CPU) are allocated to a process, the process holds the CPU till it gets terminated or it reaches awaiting state. In case of non-preemptive scheduling does not interrupt a process running CPU in middle of the execution. Instead, it waits till the process complete its CPU burst time and then it can allocate the CPU to another process.

Algorithms based on preemptive scheduling are: First come

first serve (FCFS), Priority (non-preemptive version),SJF(non-preemptive) etc.

Scheduling Techniques

Different time with respect to a process

- **Arrival Time(AT):** Time at which the process arrives in the ready queue.
- **Burst Time(BT):** Time required by a process for CPU execution.
- **Completion Time(CT):** Time at which process completes its execution.
- **Turn Around Time(TAT):** The time interval from the time of submission of a process to the time of the completion of the process.

  OR,

  Time Difference between completion time and arrival time.

  i.e.**TAT= CT - AT**
- **Waiting Time(WT):** The total time waiting in a ready queue.

  OR**,**

  Time Difference between turnaround time and burst time.

  i.e.**WT = TAT - BT**
- **Response time(RT):** Amount of time from when a

request was submitted until the first response is produced.

- **Quantum size:** It is the specific time interval used to prevent any one process monopolizing the system i.e. can be run exclusively.

A Process Scheduler schedules different processes to be assigned to the CPU based on particular scheduling algorithms**. The rule or set of rules that is followed to schedule the process is known as process scheduling algorithm.** The algorithms are either non-preemptive or preemptive**.**

Some of the algorithms are:

i. **First Come First Serve (FCFS)Scheduling**

- It is a non-preemptive scheduling algorithm.
- The process which arrives first, gets executed first or the process which requests the CPU first, gets the CPU allocated first.
- First Come First Serve, is just like FIFO (First in First out) Queue data structure, where the data element which is added to the queue first, is the one who leaves the queuefirst.

A perfect real-life example of FCFS scheduling is **buying tickets at ticket counter**.

Advantages

- FCFS algorithm doesn't include any complex logic, it just puts the process requests in a queue and executes it one by one. Hence, FCFS is pretty simple and easy toimplement.
- Eventually, every process will get a chance to run, so starvation doesn'toccur.

Disadvantages

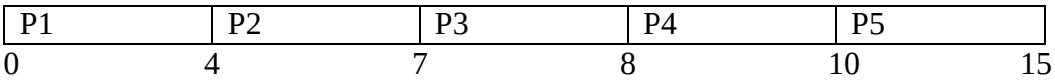
- There is no option for pre-emption of a process. If a process is started, then CPU executes the process until it ends.
- Because there is no pre-emption, if a process executes for a long time, the processes in the back of the queue will have to wait for a long time before they get a chance to be executed.

Example 1: Consider the following set of processes having their arrival time and CPU-burst time. Find the average waiting time and turnaround time using FCFS.

Process	Arrival Time (ms)	Burst Time (ms)
P1	0	4
P2	1	3
P3	2	1
P4	3	2
P5	4	5

Solution:

Preparing Gantt chart using FCFS we get,



We know,

TAT (Turnaround Time) = CT (Completion Time) – AT (Arrival Time)

and WT (Waiting Time) = TAT (Turnaround Time) – BT (Burst Time)

Now finding TAT and WT,

Process	AT	BT	CT	TAT	WT
P1	0	4	4	4	0
P2	1	3	7	6	3
P3	2	1	8	6	5
P4	3	2	10	7	5
P5	4	5	15	11	6
				∑TAT= 34	**∑WT= 18**

Average Turnaround time(ATAT) = $\sum$**TAT/N** = 34/ 5 = 6.8 ms

Average Waiting Time (AWT) = $\sum$**WT/ N** = 18/5 = 3.6 ms

Note: We can also find throughput and CPU utilization of the system using formula,

Throughput = (Number of process executed / Total time required) = 5/15 = 0.33

CPU Utilization = (Total working time / Total time required) = 15/15 = 1 (100 %)

Example 2: Consider the given processes with arrival and burst time given below. Schedule the given processes using FCFS and find average waiting time and average turn-around time.

PROCESSES	ARRIVAL TIME	BRUST TIME
A	0	3
B	2	6
C	4	4
D	6	5

E	8	2
F	3	4

SOLUTION:

Applying FCFS Gantt chart for the given processes becomes:

A	B	F	C	D	E

0 3 9 13 17 22 24

Now, from Gantt chart,

Process	Arrival time (AT)	Burst time (BT)	Completion time (CT)	Turnaroundtime TAT = CT – AT	Waiting time WT=TAT-BT
A	0	3	3	3	0
B	2	6	9	7	1
C	4	4	13	9	5
D	6	5	18	12	7
E	8	2	20	12	10
F	3	4	24	21	17
				∑ **TAT=64**	∑ **WT=40**

Average turn-around time **=** ∑(TAT) / n = 64/6= 10.67

Average waiting time = ∑(WT) / n =40/6 = 6.67

Example 3: Consider the processes P1, P2, P3, P4 given in the below table, arrives for execution in the same order, with Arrival Time 0, and given Burst Time, find the average waiting time using the FCFS scheduling algorithm.

Process	**Burst Time (ms)**
P1	21
P2	3
P3	6
P4	2

Solution:

P1	P2	P3	P4

0 21 24 30 32

Now from Gantt chart,

Process	Arrival Time (ms)	Burst Time	Completion Time	Turnaround Time TAT=CT-AT	Waiting Time WT = TAT - BT
P1	0	21	21	21	0
P2	0	3	24	24	21
P3	0	6	30	30	24
P4	0	2	32	32	30

Average WT = (0+21+24+30)/4 = 18.75

i. Shortest Job First (SJF)Scheduling

- It is a non-preemptive schedulingalgorithm.
- It is also known as shortest job next(SJN).
- It is a scheduling policy that selects the waiting process with the smallest execution time to executenext.

Advantages.

- The throughput is increased because more processes can be executed in less amount of time.
- Having minimum average waiting time among all schedulingalgorithms.

Disadvantages:

- ItispracticallyinfeasibleasOperatingSystemmaynotknowbursttimeandthereforemay not sortthem.
- Longer processes will have more waiting time, eventually they'll sufferstarvation.

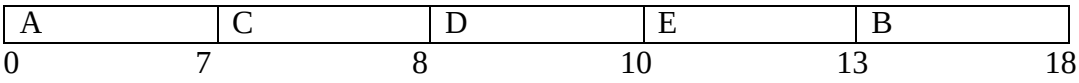
Example 1: Consider the given processes with arrival and burst time given below. Schedule the given processes using SJF and find average waiting time (AWT) and average turnaround time(ATAT).

PROCESSES	ARRIVAL TIME	BRUST TIME
A	0	7

B	2	5
C	3	1
D	4	2
E	5	3

Solution:

According to the given question the Gantt chart for SJF given processes becomes:



Now,

Calculating of TAT and WT using: **TAT= CT – AT and WT = TAT - BT**

Process	**Arrival time**	**Burst time**	**Completion time**	**Turn-around time**	**Waiting time**
A	0	7	7	7	0
B	2	5	18	16	11
C	3	1	8	5	4
D	4	2	10	6	4
E	5	3	13	8	5
				∑TAT=42	∑WT=24

Average turn-around time = ∑(TAT)/n= 42/5 = 10.83

Average waiting time = ∑(WT) / n =24/5 =4.8

Example 2: Five processes P1,P2,P3,P4 and P5 having burst time 5,3,4,3,1 arrives at CPU . Find the Average TAT and Average WT using Shortest job first scheduling algorithm.

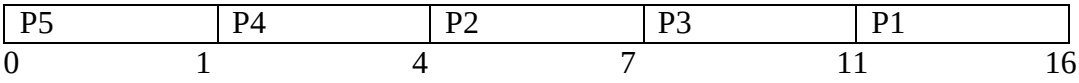
PROCESSES	BRUST TIME (ms)
P1	5
P2	3
P3	4
P4	3
P5	1

Solution

Since there is no any arrival time so it is assumed all the

processes arrive initially. i.e. AT=0

The Gantt chart for the above processes becomes:



Now, using Gantt chart above and formula,

TAT = (CT-AT) and WT = (TAT-BT)

Process	Arrival time	Burst time	CT	TAT	WT
P1	0	5	16	16	11
P2	0	3	7	7	4
P3	0	4	11	11	7
P4	0	3	4	4	1
P5	0	1	1	1	0
				∑TAT=39	∑WT=23

Average turn-around time = ∑(TAT)/n = 39/5 = 7.8 ms

Average waiting time = ∑(WT) / n =23/5 = 4.6ms

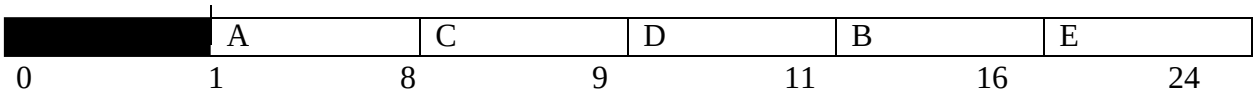
Example 3: Consider the given processes with arrival and burst time given below. Schedule the given processes using SJF and find waiting time and turn-around time.

PROCESSES	ARRIVAL TIME	BRUST TIME
A	1	7
B	2	5
C	3	1
D	4	2
E	5	8

Solution

The Gantt chart for the above processes becomes:

Here no process arrives at time= 0, so CPU stay in idle position.



Now, using Gantt chart

above and formula,

TAT = (CT-AT) and

WT = (TAT-BT)

Process	Arrival time	Burst time	CT	TAT	WT
A	1	7	8	7	0
B	2	5	16	14	9
C	3	1	9	6	5
D	4	2	11	7	5
E	5	8	24	19	11

Here Throughput = (5/24) and CPU Utilization=(23/24) since CPU stay in idle for 1unit oftime.

i. Shortest Remaining Time (SRT)Scheduling

- It is preemptive scheduling algorithm
- It is also called Preemptive shortest job first or Shortest remaining time next or Shortest remaining time first
- In this scheduling algorithm, the process with the smallest amount of time remaining until completion is selected to execute. Since the currently executing process is the one with the shortest amount of time remaining by definition, and since that time should only reduce as execution progresses, processes will always run until they complete or a new process is added that requires a smaller amount of time.

Advantage:

- Short processes are handled very quickly.
- The system also requires very little overhead since it only makes a decision when a process completes or a new process is added.
- When a new process is added, the algorithm only needs to compare the currently executing process with the new process, ignoring all other processes currently waiting to execute.

Disadvantage:

- Like shortest job first, it has the potential for process starvation.
- Long processes may be held off indefinitely if short processes are continually added.

Example1:Considerthefollowingprocesseswitharrivaltimeandbursttime.Schedulethem using SRT.

PROCESSES	ARRIVAL TIME(AT)	BRUST TIME (BT)
A	0	7
B	1	5
C	2	3
D	3	1
E	4	2
F	5	1

Solution:

Gantt chart for the above processes becomes:

A	B	C	D	C	C	F	E	B	A

0 1 2 3 4 5 6 7 9 1319

Now, TAT = (CT-AT) and WT = (TAT-BT)

Process	Arrival time	Burst time	CT	TAT	WT
A	0	7	19	19	12
B	1	5	13	12	7
C	2	3	6	4	1
D	3	1	4	1	0
E	4	2	9	5	3
F	5	1	7	2	1
				∑TAT=43	**∑WT=24**

Average turn-around time = ∑(TAT)/n = 43/6 = 7.16 units
Average waiting time = ∑(WT) / n =24/6 = 4units.

ii. Priority Scheduling

- In this scheduling Priority is assigned for each process.
- Process with highest priority is executed first and soon.
- Processes with same priority are executed in FCFS manner.
- Priority can be decided based on memory requirements, time

requirements or anyother resourcerequirement

Advantages of Priority Scheduling:

- The priority of a process can be selected based on memory requirement, time requirement or user preference. For example, a high-end game will have better graphics, that means the process which updates the screen in a game will have higher priority so as to achieve better graphics performance.

Disadvantages:

- A second scheduling algorithm is required to schedule the processes which have same priority.
- Inpreemptivepriorityscheduling,ahigherpriorityprocessca nexecuteaheadofanalready executing lower priority process. If lower priority process keeps waiting for higher priority processes, starvation occurs.

Priority scheduling are of both types: preemptive and non-preemptive.

a. Non-Preemptive Priority Scheduling

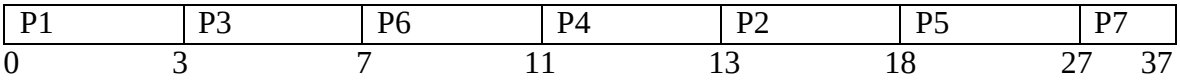
- In the Non-Preemptive Priority scheduling, The Processes are scheduled according to the priority number assigned to them.
- Once the process gets scheduled, it will run till the completion.
- **Generally, the lower the priority number, the higher is the priority of the process**.
- **Example1:**There are 7 processes P1,P2,P3,P4,P5,P6andP7. Their priorities, ArrivalTime and burst time are given in the table. Find waiting time and response time using non preemptive priority scheduling. Also find average WT. Lower the number higher the priority.

Process	Priority	Arrival Time	Burst Time
P1	2	0	3

P2	6	2	5
P3	3	1	4
P4	5	4	2
P5	7	6	9
P6	4	5	4
P7	10	7	10

Solution:

Gantt chart for the process is:



From the GANTT Chart prepared, we can determine the completion time of every process. The turnaround time, waiting time and response time will be determined.

Turn Around Time = Completion Time - Arrival Time

Waiting Time = Turn Around Time - Burst Time

Response time = Time at which process get first response – Arrival Time

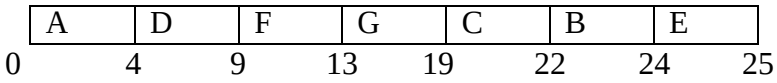
Process	**Priority**	**Arrival Time**	**Burst Time**	**Completion Time**	**Turnaround Time**	**Waiting Time**	**Response Time**
P1	2	0	3	3	3	0	0
P2	6	2	5	18	16	11	11
P3	3	1	4	7	6	2	2
P4	5	4	2	13	9	7	7
P5	7	6	9	27	21	12	12
P6	4	5	4	11	6	2	2
P7	10	7	10	37	30	20	20

Average Waiting Time = (0+11+2+7+12+2+20)/7 = 54/7 units

Example 2: Consider the given processes below. Schedule the algorithm using non-preemptive priority, schedule the given processes. Assume 12 as highest priority and 2 as lowest priority.

Process	AT	BT (ms)	PRIORITY
A	0	4	2(lowest)
B	1	2	4
C	2	3	6
D	3	5	10
E	4	1	3
F	5	4	12(highest)
G	6	6	9

Solution:



a. Preemptive PriorityScheduling

InPreemptivePriorityScheduling,atthetime ofarrivalofaprocessinthereadyqueue,itsPriority is compared with the priority of the other processes present in the ready queue as well as with the one which is being executed by the CPU at that point of time. The One with the highest priority among all the available processes will be given the CPU next.

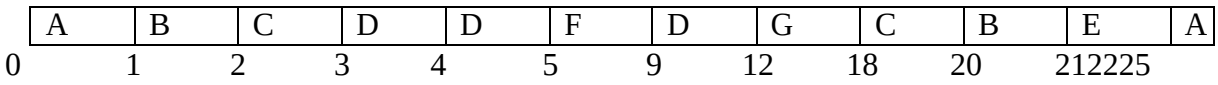
Example1:Considerthegivenprocessesbelow.Schedulethealgorithmusingpreemptivepriority algorithm and hence find AWT and ATAT. Assume 12 as highest priority and 2 as lowestpriority.

Process	AT	BT (ms)	PRIORITY
A	0	4	2(lowest)
B	1	2	4
C	2	3	6

D	3	5	10
E	4	1	3
F	5	4	12(highest)
G	6	6	9

Solution:

The Gantt chart for the given processes becomes: -



Now,

Process	AT	BT	PRIORITY	CT	TAT	WT
A	0	4	2	25	25	21
B	1	2	4	21	20	18
C	2	3	6	20	18	15
D	3	5	10	12	9	4
E	4	1	3	22	18	17
F	5	4	12	9	4	0
G	6	6	9	18	12	6
					∑TAT=106	**∑WT=81**

Average turn-around time= (∑TAT=106)/7 = 15.14 ms

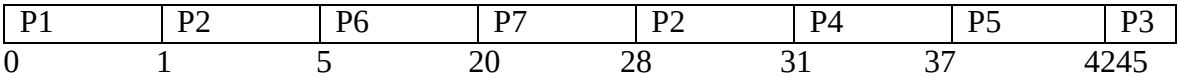
Average waiting time will be = (∑WT=81)/7 =11.57 ms

Example 2: There are 7 processes P1, P2, P3, P4, P5, P6 and P7 given. Their respective priorities, Arrival Times and Burst times are given in the table below. Find average WT using preemptive priority scheduling.

Process Id	Priority	Arrival Time	Burst Time
P1	2(L)	0	1
P2	6	1	7-1=6-1=5-1=4-1=3
P3	3	2	3
P4	5	3	6
P5	4	4	5
P6	10(H)	5	15
P7	9	15	8

Solution:

Gantt chart for above process is:



Now,

Process	Priority	Arrival Time	Burst Time	Completion Time	Turnaround Time	Waiting Time
P1	2	0	1	1	1	0
P2	6	1	7	31	30	23
P3	3	2	3	45	43	40
P4	5	3	6	37	34	28
P5	4	4	5	42	38	33
P6	10(H)	5	15	20	15	0
P7	9	6	8	28	22	14

V. Round Robin Scheduling

- CPU is assigned to the process for a fixed amount oftime.
- This fixed amount of time is called as time quantum or timeslice.
- After the time quantum expires, the running process is preempted and sent to the ready queue.
- Then, the processor is assigned to the next arrivedprocess.
- It is always preemptive schedulingalgorithm.

Advantages

- It gives the best performance in terms of average responsetime.
- It is best suited for time sharing system, client server architecture and interactivesystem.

Disadvantages

- It leads to starvation for processes with larger burst time as they have to repeat thecycle manytimes.
- Its performance heavily depends on timequantum.
- Priorities cannot be set for theprocesses.

Example 1: Consider the following processes with ATand BT

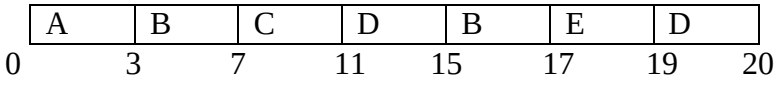
given. Find waiting time and turnaround time using round robin with time quantum = 4**.**

PROCESSES	ARRIVAL TIME(AT)	BURST TIME (BT)
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Solution:

Ready Queue: ABCDBED

Gantt chart:



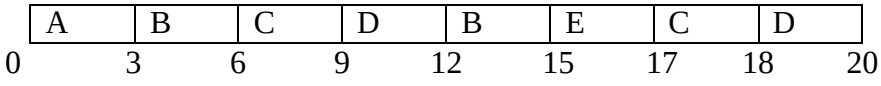
Now,

Process	Arrival time	Burst time	CT	TAT	WT
A	0	3	3	3	0
B	2	6	17	15	9
C	4	4	11	7	2
D	6	5	20	14	10
E	8	2	19	11	9

Example 2: Draw the gantt chart of above processes using round robin with quantum 3 and 2. Solution:

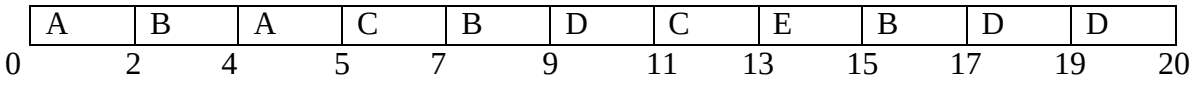
TQ=3

Ready queue:ABCDBECD



Now, TQ = 2

Ready queue:ABACBDCEBDD



i. **HRRN (Highest Response RatioNext)**

- Highest Response Ratio Next (HRNN) is one of the most optimal schedulingalgorithms.
- This is a **non-preemptive algorithm**in which, the scheduling is done on the basis of an extra parameter called ResponseRatio.
- A Response Ratio is calculated for each of the available jobs and the Job with the highest response ratio is given priority over theothers.

Response Ratio is calculated by the given formula, Response Ratio = (WT+BT)/BT

Where,
WT → Waiting Time
BT → Service Time or Burst Time

Advantages-

- It performs better than SJF Scheduling.
- It not only favors the shorter jobs but also limits the waiting time of longer jobs.

Disadvantages-

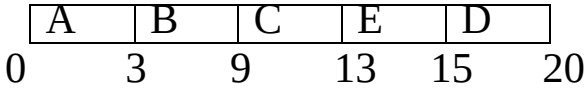
- It cannot be implemented practically. This is because burst time of the processes cannot be known in advance.

Example: Consider the following processes with AT and BT given. Find waiting time and turnaround time using HRRN.

PROCESSES	ARRIVAL TIME(AT)	BRUST TIME (BT)
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Solution:

Gantt chart:



Here A and B can be executed in FIFO. At, time = 9 C,D,E arrives so RR need to be calculated.

Response Ratio of C $(RR)_C$= (5+4)/4 = 2.25
Response Ratio of D$(RR)_D$= (3+5)/5 = 1.6
Response Ratio of E$(RR)_E$= (1+2)/2 = 1.5

Since $(RR)_C$is larger than other hence C runs.

Response Ratio of D$(RR)_D$= (7+5)/5 = 2.4
Response Ratio of E$(RR)_E$= (5+2)/2 = 3.5

Since $(RR)_E$is larger than other hence E runs

Now from the Gantt chart:

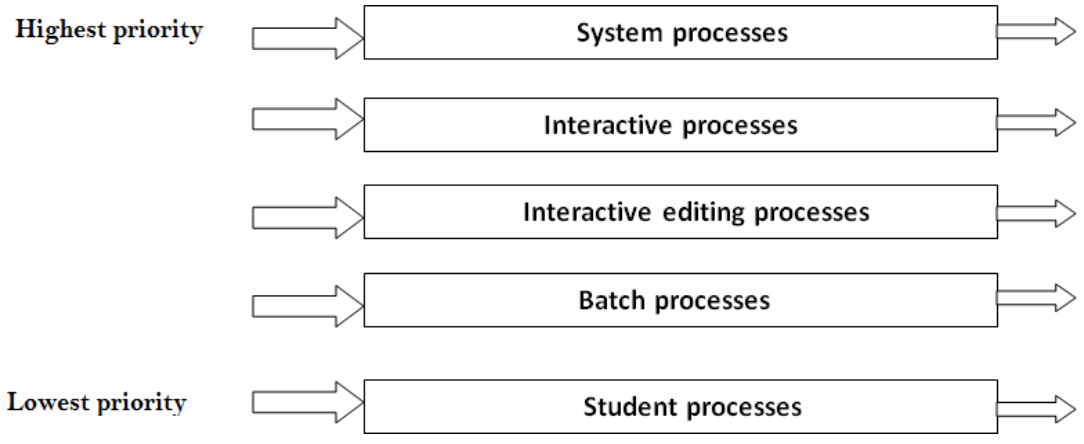
Process	Arrival time (AT)	Burst time (BT)	Completion time (CT)	Turnaroundtime TAT = CT – AT	Waiting time WT=TAT-BT
A	0	3	3	3	0
B	2	6	9	7	1
C	4	4	13	9	5
D	6	5	20	14	9
E	8	2	15	7	5

Multilevel Queue

- In the multilevel queue scheduling algorithm, the ready queue has divided into separate queues. Based on some priority of the process; like memory size, process priority, or process type these processes are permanently assigned to one queue.
- Each queue has its own scheduling algorithm. For example, some queues are used for the foreground process and some for the back ground process.
- The foreground queue can be scheduled by using a round-robin algorithm while the background queue is scheduled by a first come first serve algorithm.

Let us take an example of a multilevel queue scheduling algorithm with five queues:

- System process
- Interactive processes
- Interactive editing processes
- Batch processes
- Student processes



Multilevel Feedback Queue

- In multilevel queue scheduling algorithm, processes are permanently stored in one queue in the system and do not move between the queue.
- There is some separate queue for foreground or background processes but the processes do not move from one queue to another queue and these processes do not change their foreground or background nature, these types of arrangement have the advantage of low scheduling but it is inflexible in nature.
- Multilevel feedback queue scheduling, it allows a process to move between the queues. These processes are separated with different CPU burst time.
- If a process uses too much CPU time, then it will be moved to the lowest priority queue.
- This idea leaves I/O bound and interactive processes in the higher priority queue.
- Similarly, the process which waits too long in a lower priority queue may be moved to a higher priority queue. This form of aging prevents starvation.

The multilevel feedback queue scheduler has the following parameters:

- The number of queues in the system.

- The scheduling algorithm for each queue in the system.
- The method used to determine when the process is upgraded to a higher-priority queue.
- The method used to determine when to demote a queue to a lower - priority queue.
- The method used to determine which process will enter in queue and when that process needs service.

Thread Scheduling

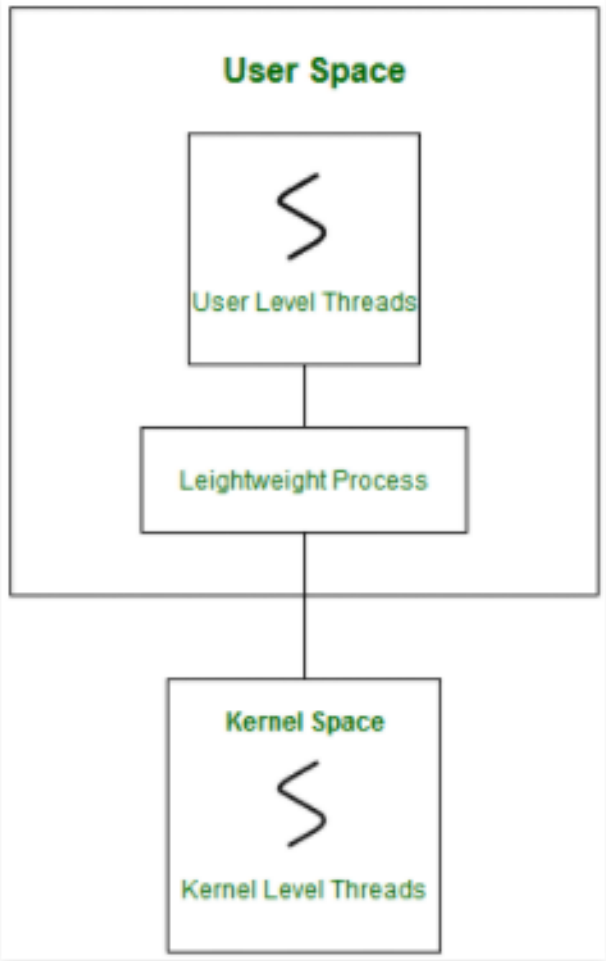
Many computer configurations have a single CPU. Hence, threads run one at a time in such a way as to provide an illusion of concurrency. Execution of multiple threads on a single CPU in some order is called *scheduling*.

Scheduling of threads involves two boundaries scheduling,

- Scheduling of user level threads (ULT) to kernel level threads (KLT) via lightweight process (LWP) by the application developer.
- Scheduling of kernel level threads by the system scheduler to perform different unique OS functions.

Lightweight Process (LWP):

Light-weight process are threads in the user space that acts as an interface for the ULT to access the physical CPU resources. Thread library schedules which thread of a process to run on which LWP and how long. The number of LWP created by the thread library depends on the type of application.



In real-time, the first boundary of thread scheduling is beyond specifying the scheduling policy and the priority. It requires two controls to be specified for the User level threads: Contention scope, and Allocation domain. These are explained as following below.

1. Contention Scope:
The word contention here refers to the competition or fight among the User level threads to access the kernel resources. Thus, this control defines the extent to which contention takes place. It is defined by the application developer using the thread library. Depending upon the

extent of contention it is classified as Process Contention Scope and System Contention Scope.

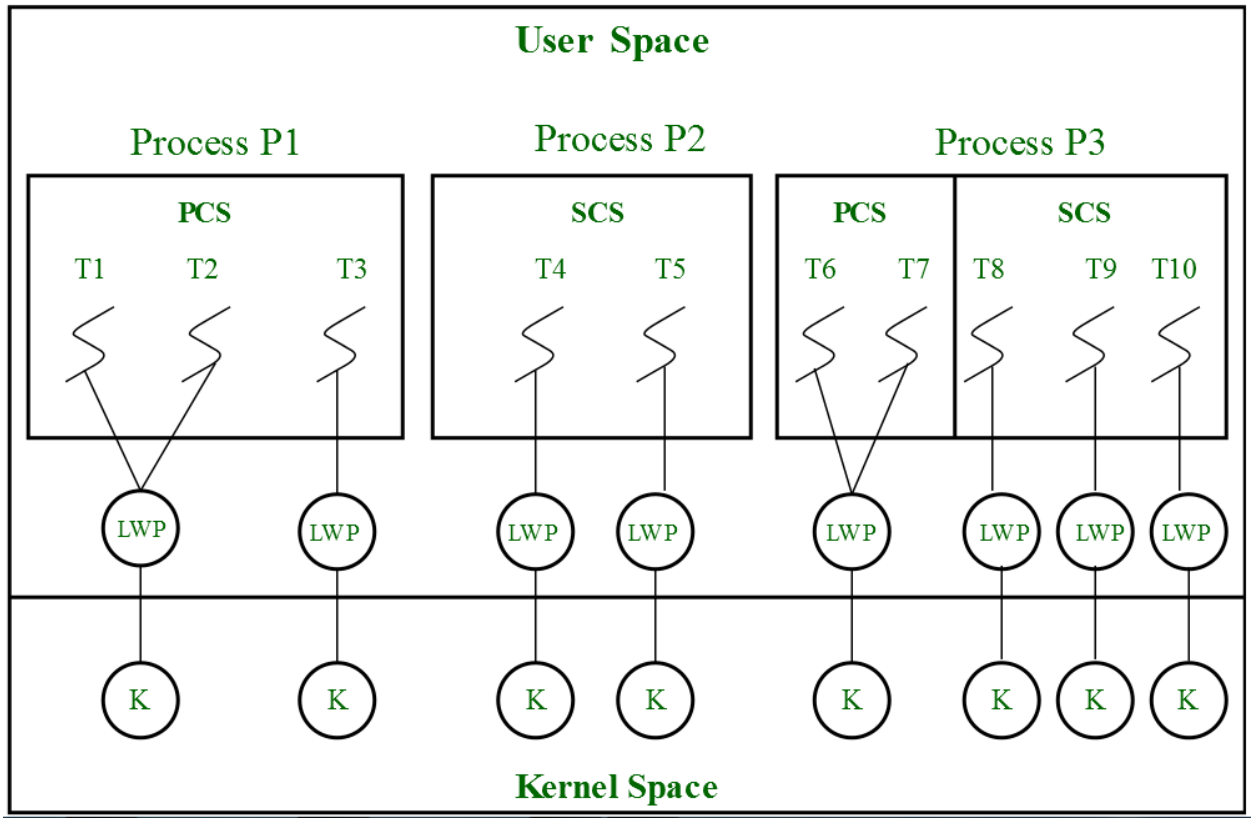
1. **Process Contention Scope (PCS) –**
   The contention takes place among threads within a same process. The thread library schedules the high-prioritized PCS thread to access the resources via available LWPs (priority as specified by the application developer during thread creation).
2. **System Contention Scope (SCS) –**
   The contention takes place among all threads in the system. In this case, every SCS thread is associated to each LWP by the thread library and are scheduled by the system scheduler to access the kernel resources.

   In LINUX and UNIX operating systems, the **POSIX Pthread** library provides a function **Pthread_attr_setscope** to define the type of contention scope for a thread during its creation.

2. Allocation Domain:

The allocation domain is **a set of one or more resources** for which a thread is competing. In a multi-core system, there may be one or more allocation domains where each consists of one or more cores. One ULT can be a part of one or more allocation domain. Due to this high complexity in dealing with hardware and software architectural interfaces, this control is not specified. But by default, the multi-core system will have an interface that affects the allocation domain of a thread.

Consider a scenario, an operating system with three process P1, P2, P3 and 10 user level threads (T1 to T10) with a single allocation domain. 100% of CPU resources will be distributed among all the three processes. The amount of CPU resources allocated to each process and to each thread depends on the contention scope, scheduling policy and priority of each thread defined by the application developer using thread library and also depends on the system scheduler. These User level threads are of a different contention scope.



In this case, the contention for allocation domain takes place as follows,

1. **Process P1:**
   All PCS threads T1, T2, T3 of Process P1 will compete among themselves. The PCS threads of the same process can share one or more LWP. T1 and T2 share an LWP and T3 are allocated to a separate LWP. Between T1 and T2 allocation of kernel resources via LWP is based on preemptive priority scheduling by the thread library. A Thread with a high priority will preempt low priority threads. Whereas, thread T1 of process p1 cannot preempt thread T3 of process p3 even if the priority of T1 is greater than the priority of T3. If the priority is equal, then the allocation of ULT to available LWPs is based on the scheduling policy of threads by the system scheduler(not by

thread library, in this case).

2. **Process P2:**
   Both SCS threads T4 and T5 of process P2 will compete with processes P1 as a whole and with SCS threads T8, T9, T10 of process P3. The system scheduler will schedule the kernel resources among P1, T4, T5, T8, T9, T10, and PCS threads (T6, T7) of process P3 considering each as a separate process. Here, the Thread library has no control of scheduling the ULT to the kernel resources.

3. **Process P3:**
   Combination of PCS and SCS threads. Consider if the system scheduler allocates 50% of CPU resources to process P3, then 25% of resources is for process scoped threads and the remaining 25% for system scoped threads. The PCS threads T6 and T7 will be allocated to access the 25% resources based on the priority by the thread library. The SCS threads T8, T9, T10 will divide the 25% resources among themselves and access the kernel resources via separate LWP and KLT. The SCS scheduling is by the system scheduler.

Concurrent process

Two processes are said to be concurrent if their execution can overlap in the time i.e. the execution of second process starts before the first completes. Typically, the processor will be executing instructions for one program while another (or several other) program(s) are waiting for I/O from external devices or from an end-user.

Parallel processing

Simultaneous use of more than one CPU or processors to execute a program or multiple computation threads. **Parallel Processing Systems** are designed to speed up the execution of programs by dividing the program into multiple fragments and processing these fragments simultaneously. Such systems are multiprocessor systems also known as tightly coupled systems. Parallel systems deal with the simultaneous use of multiple computer resources that can include a single computer with multiple processors.

Inter Process Communication (IPC)

A process can be of two type:

- Independent process.

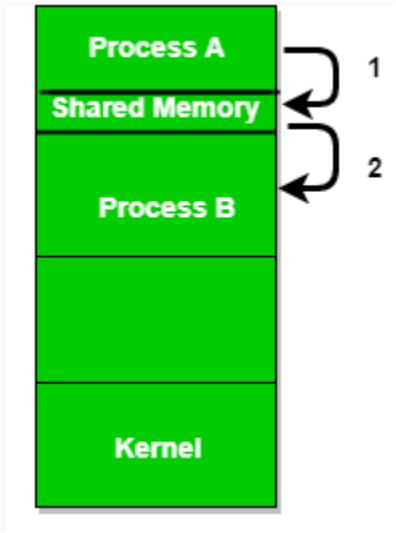
- Co-operating process.

An independent process is not affected by the execution of other processes while a co-operating process can be affected by other executing processes. Though one can think that those processes, which are running independently, will execute very efficiently but in practical, there are many situations when co-operative nature can be utilized for increasing computational speed, convenience and modularity. Inter process communication (IPC) is a mechanism which allows processes to communicate each other and synchronize their actions. The communication between these processes can be seen as a method of co-operation between them. Processes can communicate with each other using these two ways:

1. Shared Memory
2. Message passing

An operating system can implement both method of communication. First, we will discuss the shared memory method of communication and then message passing. Communication between processes using shared memory requires processes to share some or memory. One way of communication using shared memory can be imagined like this: Suppose process1 and process2 are executing simultaneously and they share some resources or use some information from other process, process1 generate information about certain computations or resources being used and

keeps it as a record in shared memory. When process2 need to use the shared information, it will check in the record stored in shared memory and take note of the information generated by process1 and act accordingly. Processes can use shared memory for extracting information as a record from other process as well as for delivering any specific information to other process.



Messaging Passing Method

In this method, processes communicate with each other without using any kind of shared memory. If two processes p1 and p2 want to communicate with each other, they proceed as follow:

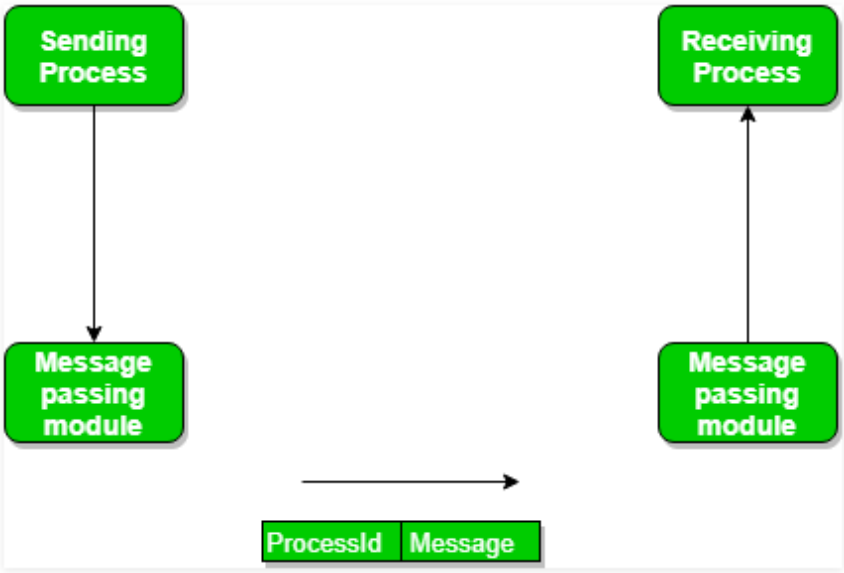
- Establish a communication link (if a link already exists, no need to establish it again.)

- Start exchanging messages using basic primitives.

  We need at least two primitives:

  – **send** (message, destination) or **send**(message)

  – **receive** (message, host) or **receive**(message)

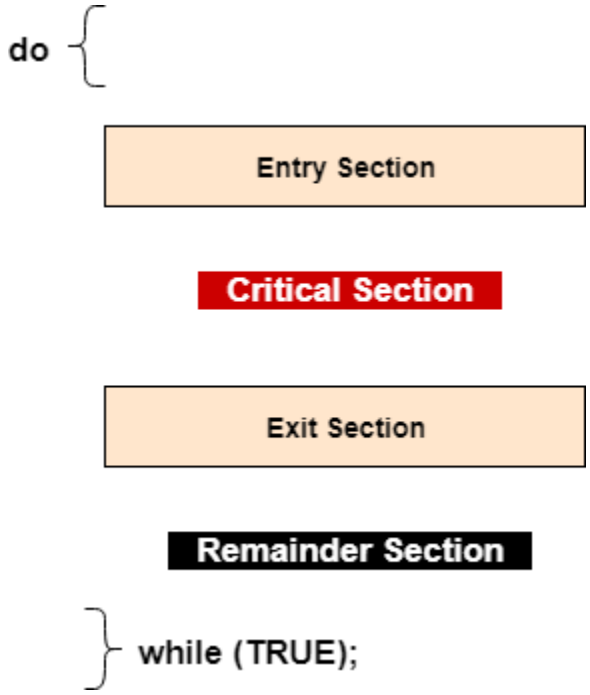


Critical section

The critical section is a code segment where the shared variables can be accessed. An atomic action is required in a critical section i.e. only one process can execute in its critical section at a time. All the other processes have to wait to execute in their critical sections.

A diagram that demonstrates the critical section is as follows:

:



In the above diagram, the entry section handles the entry into the critical section. It acquires the resources needed for execution by the process. The exit section handles the exit from the critical section. It releases the

resources and also informs the other processes that the critical section is free.

Solution to the Critical Section Problem

The critical section problem needs a solution to synchronize the different processes. The solution to the critical section problem must satisfy the following conditions:

1. **Mutual Exclusion**

   Mutual exclusion implies that only one process can be inside the critical section at any time. If any other processes require the critical section, they must wait until it is free.

2. **Progress**

   Progress means that if a process is not using the critical section, then it should not stop any other process from accessing it. In other words, any process can enter a critical section if it is free.

3. **Bounded Waiting**

   Bounded waiting means that each process must have a limited waiting time. It should not wait endlessly to access the critical section.

Race Condition

A race condition is a situation that may occur inside a critical section. This happens when the result of multiple thread execution in critical section differs according to the order in which the threads execute.

Race conditions in critical sections can be avoided if the critical section is treated as an atomic instruction. Also, proper thread synchronization using locks or atomic variables can prevent race conditions.

Mutual exclusion

A way of making sure that if one process is using a shared modifiable data, the other processes will be excluded from doing the same thing. Formally, while one process executes the shared variable, all other processes desiring to do so at the same time moment should be kept waiting; while that process has finished executing the shared variable, one of the processes waiting to do so should be allowed to proceed. In this fashion, each process executing the shared data (variables) excludes all others from doing so simultaneously. This is called Mutual Exclusion.

Mutual Exclusion Conditions

If we could arrange matters such that no two processes were ever in their critical sections simultaneously, we could avoid race conditions. We

need four conditions to hold to have a good solution for the critical section problem (mutual exclusion).

- No two processes may at the same moment inside their critical sections.
- No assumptions are made about relative speeds of processes or number of CPUs.
- No process should outside its critical section should block other processes.
- No process should wait arbitrary long to enter its critical section.

Peterson’s solution to critical section problem

- Peterson's solution is a classic software-based solution to the critical-section problem.
- It provides a good algorithmic description of solving the critical-section problem and illustrates some of the complexities involved in designing software that addresses the requirements of mutual exclusion, progress, and bounded waiting requirements.
- Peterson's solution is restricted to two processes that alternate execution between their critical sections and remainder sections.
- Peterson's solution requires two data items to be shared between the two processes:

int turn;

boolean flag[2]

where initially flag[i]=flag[j]=false

- The variable turn indicates whose turn it is to enter its critical section. That is, if **turn == i, then process P_i is allowed to execute in its critical section**.

- The flag array is used to indicate if a process is ready to enter its critical section. For example, if **flag[i] is true**, this value indicates that **Pi is ready to enter its critical section**.

P_i	P_j
do {	**do {**
flag[i]==true;	**flag[j]==true;**
turn=j;	**turn=i;**
while (flag[j] && turn==j);	**while (flag[i] && turn==i);**
/* critical section */	**/* critical section */**
flag[i]=false;	**flag[j]=false;**
/* remainder section */	**/* remainder section */**
}	**}**
while (true);	**while (true);**

Disadvantage

- Peterson's solution works for two processes, but this solution is best scheme in user mode for critical section.
- This solution is also a busy waiting solution so CPU time is wasted. So that **"SPIN LOCK"** problem can come. And this problem can come in any of the busy waiting solution.

Test and set lock

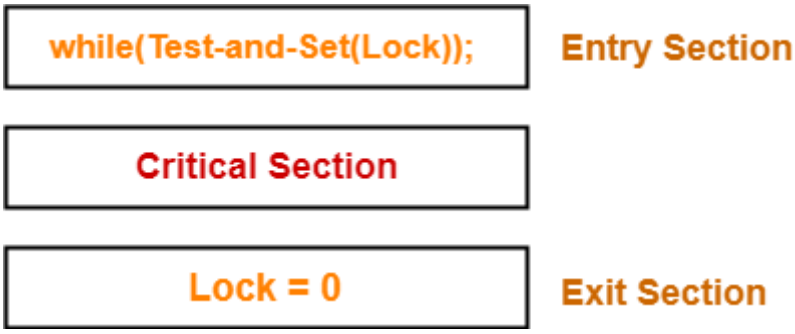
- Test and Set Lock (TSL) is a synchronization mechanism.
- It uses a test and set instruction to provide the synchronization among the processes executing concurrently.

Test-and-Set Instruction

It is an instruction that returns the old value of a memory location and sets the memory location value to 1 as a single atomic operation.

If one process is currently executing a test-and-set, no other process is allowed to begin another test-and-set until the first process test-and-set is finished.

It is a hardware solution.



The assembly code of the solution will look like following.

Enter region: // before entering its critical section process calls enter region

1. TSL R0, Lock // copy lock variable to register and set lock to 1
2. CMP R0, #0 // is lock variable 0
3. JNZ step 1 // if it not zero lock is set so loop
4. RET // return to caller; critical section entered

Leave region: // calls leave region when a process wants to leave a critical section

Move Lock, #0 // store 0 in lock variable

RET // return to caller

Initially, lock value is set to 0.

- Lock value = 0 means the critical section is currently vacant and no process is present inside it.

- Lock value = 1 means the critical section is currently occupied and a process is present inside it.

Scene-01:

- Process P_0 arrives.
- It executes the test-and-set (Lock) instruction.
- Since lock value is set to 0, so it returns value 0 to the while loop and sets the lock value to 1.
- The returned value 0 breaks the while loop condition.
- Process P0 enters the critical section and executes.
- Now, even if process P0 gets preempted in the middle, no other process can enter the critical section.
- Any other process can enter only after process P0 completes and sets the lock value to 0.

Scene-02:

- Another process P1 arrives.
- It executes the test-and-set (Lock) instruction.

- Since lock value is now 1, so it returns value 1 to the while loop and sets the lock value to 1.
- The returned value 1 does not break the while loop condition.
- The process P1 is trapped inside an infinite while loop.
- The while loop keeps the process P1 busy until the lock value becomes 0 and its condition breaks.

Scene-03:

Process P0 comes out of the critical section and sets the lock value to 0.

- The while loop condition breaks.
- Now, process P1 waiting for the critical section enters the critical section.
- Now, even if process P1 gets preempted in the middle, no other process can enter the critical section.
- Any other process can enter only after process P1 completes and sets the lock value to 0.

Characteristics-

The characteristics of this synchronization mechanism are-

- It ensures mutual exclusion.
- It is deadlock free.
- It does not guarantee bounded waiting and may cause starvation.
- It suffers from spin lock.
- It is not architectural neutral since it requires the operating system to support test-and-set instruction.
- It is a busy waiting solution which keeps the CPU busy when the process is actually waiting.

Let's examine TSL on the basis of the four conditions.

- **Mutual Exclusion**

  Mutual Exclusion is guaranteed in TSL mechanism since a process can never be preempted just before setting the lock variable. Only one process can see the lock variable as 0 at a particular time and that's why, the mutual exclusion is guaranteed.

- **Progress**

  According to the definition of the progress, a process which doesn't want to enter in the critical section should not stop other processes to get into it. In TSL mechanism, a process will execute the TSL instruction only when it wants to get into the critical section. The value of the lock will always be 0 if no process doesn't want to enter into the critical section hence the progress is always guaranteed in TSL.

- **Bounded Waiting**

  Bounded Waiting is not guaranteed in TSL. Some process might not get a chance for so long. We cannot predict for a process that it will definitely get a chance to enter in critical section after a certain time.

- **Architectural Neutrality**

  TSL doesn't provide Architectural Neutrality. It depends on the hardware platform. The TSL instruction is provided by the operating system. Some platforms might not provide that. Hence it is not Architectural natural.

Lock variable

This is the simplest synchronization mechanism. This is a Software Mechanism implemented in User mode. This is a busy waiting solution which can be used for more than two processes.

Consider having a single, shared (lock) variable, initially 0. When a process wants to enter its critical region, it first tests the lock. If the lock is 0, the process sets it to 1 and enters the critical region. If the lock is already 1, the process just waits until it becomes 0. Thus, a 0 means that no process is in its critical region, and a 1 means that some process is in its critical region.

The pseudo code of the mechanism looks like following.

```
Entry Section →
While (lock! = 0);
Lock = 1;
//Critical Section
Exit Section →
Lock =0;
```

Unfortunately, this idea contains flaw that violates mutual exclusion. Suppose that one process reads the lock and sees that it is 0. Before it can set the lock to 1, another process is scheduled, runs, and sets the lock to 1. When the first process runs again, it will also set the lock to 1, and two processes will be in their critical regions at the same time.

Here’s the assembly code of the lock implementation

1. Load R0, Lock

2. CMP R0, #0

3. JNZ Step 1

4. Store Lock, #1

5. Critical section

6. Store Lock, #0

What are Threads?

Thread is an execution unit which consists of **process id (Pid), its own program counter, a stack, and a set of registers.** Threads are also known as Lightweight processes.Threads have same properties as of the process so they are called as light weight processes. For example, in a browser, multiple tabs can be different threads. MS Word uses multiple threads: back ground thread may check spelling and grammar, while the foreground thread process inputs, while yetone thread to loads images from the hard drives, and a fourth does specific periodic automatic backup of the file being edited, etc. Threads are executed one after another but gives the illusion as if they are executing in parallel. Each thread has different states. Each thread has

1. Process ID
2. A program Counter
3. A register set
4. A stack space

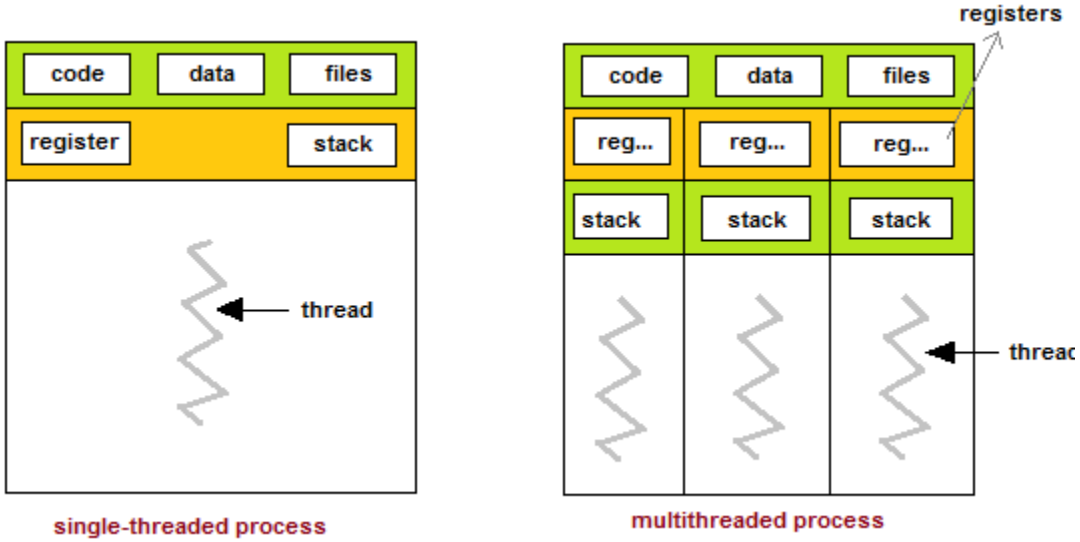


Fig: Threads

Advantages/Usages of Thread over Process

1. Responsiveness: If the process is divided into multiple threads, if one thread completes its execution, then its output can be immediately returned.

2. Faster context switch: Context switch time between threads is lower compared to process context switch. Process context switching requires more overhead from the CPU.

3. Effective utilization of multiprocessor system: If we have multiple threads in a single process, then we can schedule multiple threads on multiple processor. This will make process execution faster.

4. Resource sharing: Resources like code, data, and files can be shared among all threads within a process.

Note: stack and registers can't be shared among the threads. Each thread has its own stack and registers.

5. Communication: Communication between multiple threads is easier, as the threads shares common address space. while in process we have to follow some specific communication technique for communication between two process.

6. Enhanced throughput of the system: If a process is divided into multiple threads, and each thread function is considered as one job, then the number of jobs completed per unit of time is increased, thusincreasing the throughput of the system.

Types of Threads

There are two types of threads

- ### User level (space) threads

  It is implemented in the user level library;and the kernel is not aware of the existence of these threads. They are not created using the system calls, the user-level threads are implemented by users. Thread switching does not need to call OS and to cause interrupt to Kernel. **Kernel doesn't know about the user level thread and manages them as if they were single-threaded processes.** User-level threads are small and much faster than kernel level threads. They are represented by **a program counter(PC), stack, registers and a small process control block**. Also, there is no kernel involvement in synchronization for user-level threads.

- ### Kernel level (space) threads

  Kernel-level threads are handled by the operating system directly and the thread management is done by the kernel. The context information for the process as well as the process threads is all managed by the kernel. Because of this, kernel-level threads are slower than user-level threads.

Multi-Threading Models

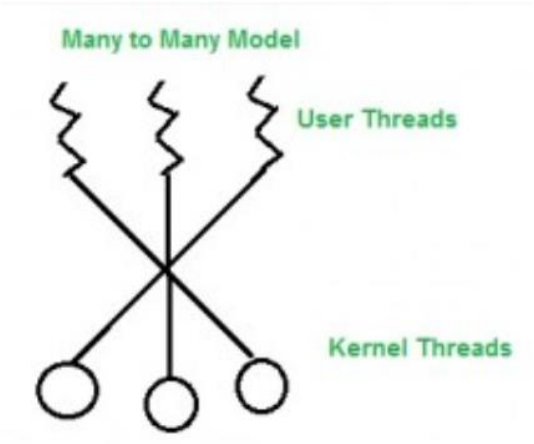
Many operating systems support kernel thread and user thread in a combined way. Example of such system is Solaris. Multi-threading model are of three types.

- Many to many
- Many to one
- One to one

1. Many to many

The many to many model maps many of the user threads to an equal number or lesser kernel threads. The number of kernel threads depends on the application or machine. There can be as many user threads as required and their corresponding kernel threads can run in parallel on a multiprocessor.

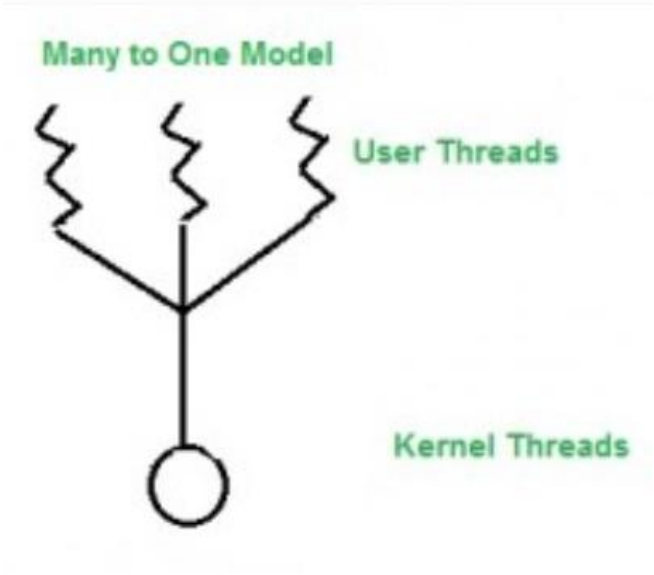
A diagram that demonstrates the “many to many” model is given as follows:



Many to many

2. Many to one

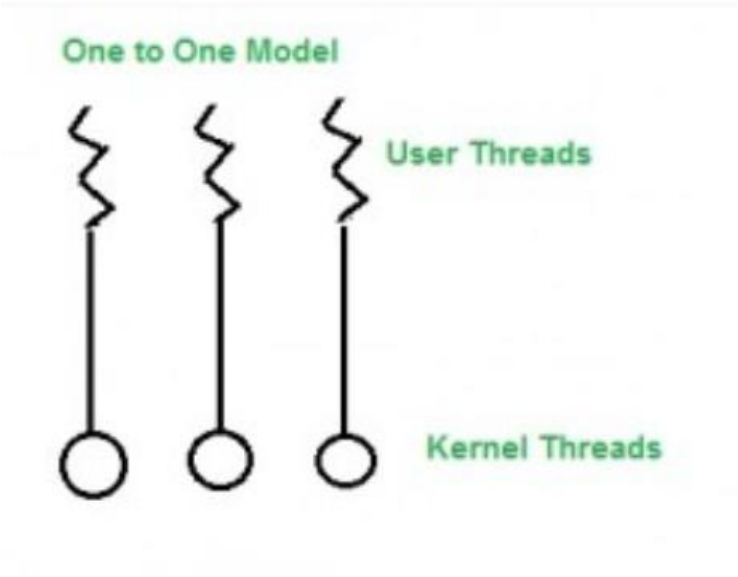
In this model, we have multiple user threads mapped to one kernel thread. In this model when a user thread makes a blocking system call entire process blocks. As we have only one kernel thread and only one user thread can access kernel at a time, so multiple threads are not able access multiprocessor at the same time.



Many to one

3. One to One

In this model, one to one relationship between kernel and user thread. In this model multiple threads can run on multiple processors. Problem with this model is that creating a user thread requires the corresponding kernel thread.



One-to-one model

Assignment2: Differentiate between thread and process.

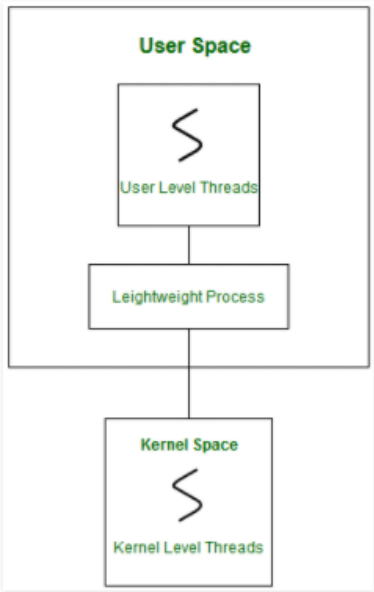
Thread Scheduling

Many computer configurations have a single CPU. Hence, threads run one at a time in such a way as to provide an illusion of concurrency. Execution of multiple threads on a single CPU in some order is called *scheduling*.

Scheduling of threads involves two boundary scheduling,

- Scheduling of user level threads (ULT) to kernel level threads (KLT) via light weight process (LWP) by the application developer.
- Scheduling of kernel level threads by the system scheduler to perform different unique OS functions.

Light weight Process (LWP):
Light-weight process are threads in the user space that acts as an interface for the ULT to access the physical CPU resources. Thread library schedules which thread of a process to run on which LWP and how long. The number of LWP created by the thread library depends on the type of application.



In real-time, the first boundary of thread scheduling is beyond specifying the scheduling policy and the priority. It requires two controls to be specified for the User level threads: Contention scope, and Allocation domain. These are explained as following below.

1. Contention Scope:
The word contention here refers to the competition or fight among the User level threads to access the kernel resources. Thus, this control defines the extent to which contention takes place. It is defined by the application developer using the thread library. Depending upon the

extent of contention it is classified as Process Contention Scope and System Contention Scope.

3. **Process Contention Scope (PCS) –**
   The contention takes place among threads within a same process. The thread library schedules the high-prioritized PCS thread to access the resources via available LWPs (priority as specified by the application developer during thread creation).

4. **System Contention Scope (SCS) –**
   The contention takes place among all threads in the system. In this case, every SCS thread is associated to each LWP by the thread library and are scheduled by the system scheduler to access the kernel resources.

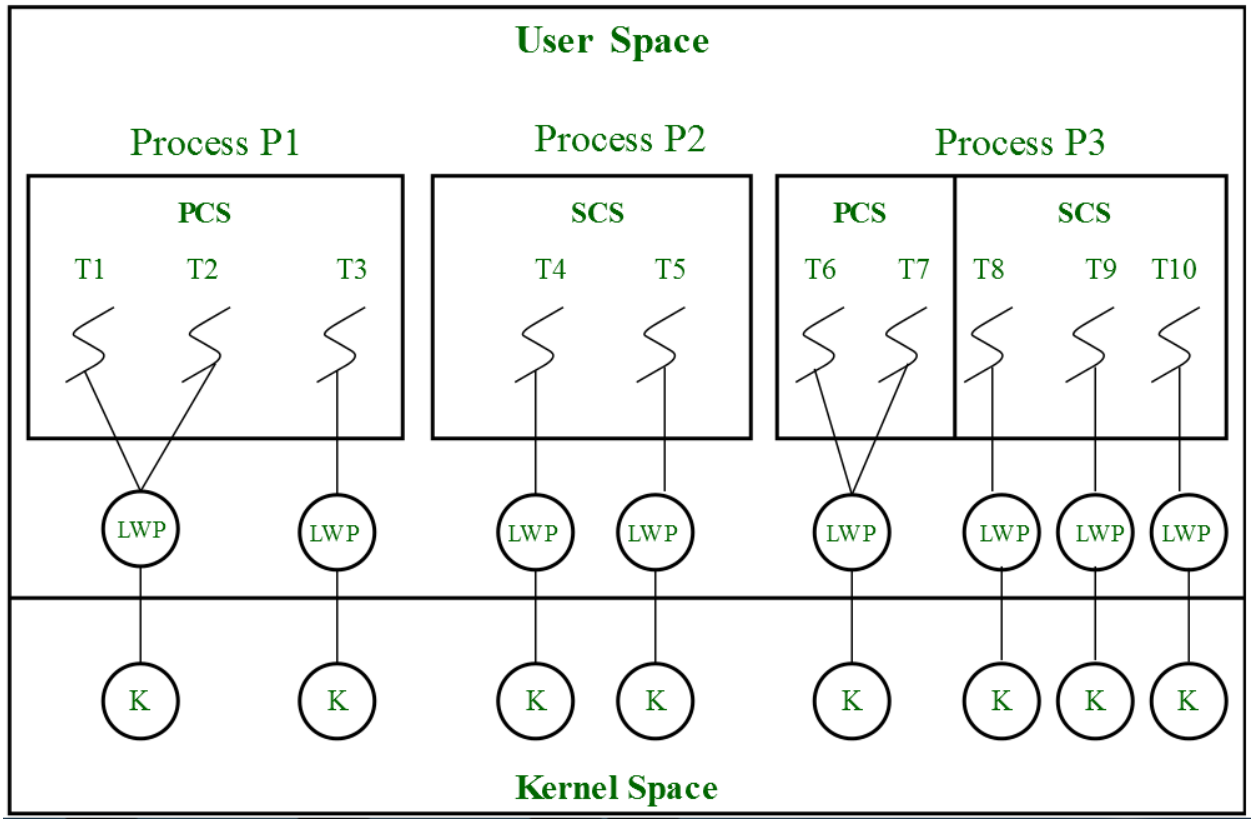
   In LINUX and UNIX operating systems, the **POSIX Pthread** library provides a function **Pthread_attr_setscope** to define the type of contention scope for a thread during its creation.

2. Allocation Domain:

The allocation domain is **a set of one or more resources** for which a thread is competing. In a multicore system, there may be one or more allocation domains where each consists of one or more cores. One ULT can be a part of one or more allocation domain. Due to this high complexity in dealing with hardware and software architectural interfaces, this control is not specified. But by default, the multicore system will have an interface that affects the allocation domain of a thread.

Consider a scenario, an operating system with three process P1, P2, P3 and 10 user level threads (T1 to T10) with a single allocation domain. 100% of CPU resources will be distributed among all the three processes. The amount of CPU resources allocated to each process and

to each thread depends on the contention scope, scheduling policy and priority of each thread defined by the application developer using thread library and also depends on the system scheduler. These User level threads are of a different contention scope.



In this case, the contention for allocation domain takes place as follows,

4. **Process P1:**
   All PCS threads T1, T2, T3 of Process P1 will compete among themselves. The PCS threads of the same process can share one or more LWP. T1 and T2 share an LWP and T3 are allocated to a separate LWP. Between T1 and T2 allocation of kernel resources via LWP is based on preemptive priority scheduling by the thread library. A Thread with a high priority will

preempt low priority threads. Whereas, thread T1 of process p1 cannot preempt thread T3 of process p3 even if the priority of T1 is greater than the priority of T3. If the priority is equal, then the allocation of ULT to available LWPs is based on the scheduling policy of threads by the system scheduler(not by thread library, in this case).

5. **Process P2:**
   Both SCS threads T4 and T5 of process P2 will compete with processes P1 as a whole and with SCS threads T8, T9, T10 of process P3. The system scheduler will schedule the kernel resources among P1, T4, T5, T8, T9, T10, and PCS threads (T6, T7) of process P3 considering each as a separate process. Here, the Thread library has no control of scheduling the ULT to the kernel resources.

6. **Process P3:**
   Combination of PCS and SCS threads. Consider if the system scheduler allocates 50% of CPU resources to process P3, then 25% of resources is for process scoped threads and the remaining 25% for system scoped threads. The PCS threads T6 and T7 will be allocated to access the 25% resources based on the priority by the thread library. The SCS threads T8, T9, T10 will divide the 25% resources among themselves and access the kernel resources via separate LWP and KLT. The SCS scheduling is by the system scheduler.

Program Threats

Operating system's processes and kernel do the designated task as instructed. If a user program made these process do malicious tasks, then it is known asProgram Threats. One of the common examples of program threat is a program installed in a computer which can store and send user credentials via network to some hacker. Following is the list of some well-known program threats.

- **Trojan Horse**–
  - A Trojan horse is a standalone malicious program which may give full control of infected PC to another PC.
  - Trojan horses may make copies of them, steal information, or harm the host computer systems.
  - The Trojan horse, at first glance will appear to be useful software but will actually do damage once installed or run on your computer.
  - Trojans are designed for different purpose like changing your desktop, adding silly active desktop icons or they can cause serious damage by deleting files and destroying information on your system.
  - Trojans are also known to create a backdoor on your computer that gives unauthorized users access to your system, possibly allowing confidential or personal information to be compromised.
  - Unlike viruses and worms, Trojans do not reproduce by infecting other files nor do they self-replicate. Means Trojan horse viruses differ from other computer viruses in that they are not designed to spread themselves.
  - Most popular Trojan horses are Netbus, Subseven, Y3K Remote Administration Tool, Back Orifice, Beast, Zeus, The Blackhole Exploit Kit, Flashback Trojan.

- **Trap Door**–
  - A trap door is a secret entry point into a program that allows someone that isaware of the trap door to gain access without going through the usual security access procedures.
  - Wecanalsosaythatitisamethodofbypassingnormalauthenticationmethods.
  - It is also known as backdoor.
  - Trap doors have been used legally for many years by programmers to debug and test programs.
  - Trap doors become threats when they are used by dishonest programmers to gain unauthorized access.
  - It is difficult to implement operating system controls for trap doors.

- **Stack & Buffer Overflow**
  - Stack-based buffer overflow exploits are likely the shiniest and most common form of exploit for remotely taking over the code execution of a process.
  - These exploits were extremely common 20 years ago, but since then, a huge amount of effort has gone into mitigating stack-based overflow attacks by operating system developers, application developers, and hardware manufacturers, with changes even being made to the standard libraries developers use.
  - In software, a stack buffer overflow or stack buffer overrun occurs when a program writes to a memory address on the program's call stack outside of the intended data structure, which is usually a fixed-length buffer.

System Threats

System threats refer to misuse of system services and network connections to put user in trouble.Systemthreatscanbeusedtolaunchprogramthreatsonacompletenetworkcalledas program attack. System threats create such an environment that operating system resources/ user files are misused. Following is the list of some well-known system threats.

- **Worm–**
  - A worm is a program which spreads usually over network connections.

- Unlike a virus which attaches itself to a host program, worms always need a host program to spread.
- Worms are not normally associated with one person computer systems.
- They are mostly found in multi-user systems such as UNIX environments.
- A classic example of a worm is Robert Morrisis Internet worm 1988.

- **Virus –**
  - A computer virus is a computer program that can replicate itself and spread from one computer to another.
  - Viruses can increase their chances of spreading to other computers by infecting files on a network file system or a file system that is accessed by other computers.
  - A computer virus is a program or piece of code that is loaded onto your computer without your knowledge and runs against your wishes. Viruses can also replicate themselves.
  - Al computer viruses are manmade.

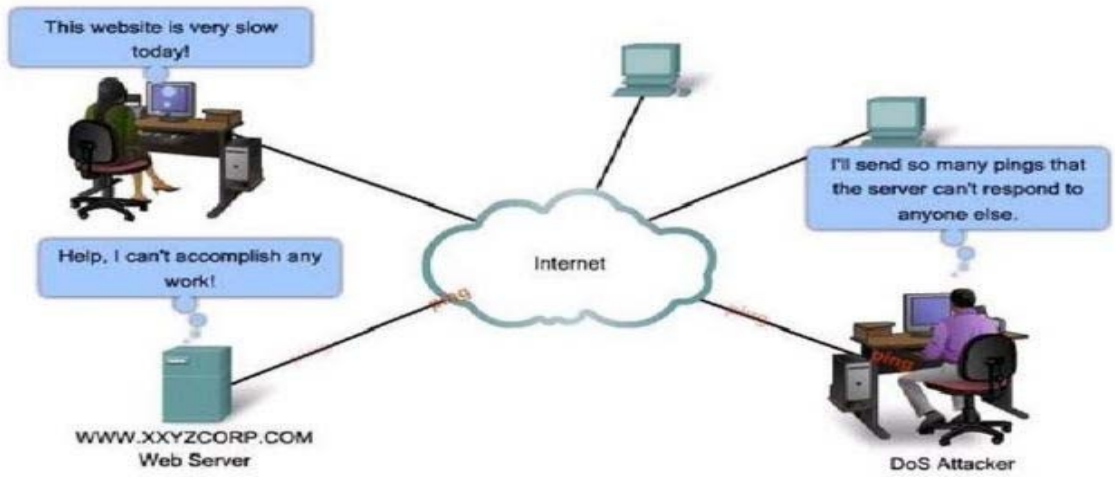
- **Denial of Service–**

  - A denial-of-service attack attempts to disable a user authentication service by flooding the service with numerous authentication attempts.
  - Denial-of-service (DoS) attack aims at disrupting the authorized use of networks, systems, or applications

    - by sending messages which exhaust service provider's resources(network bandwidth, system resources, application resources)

  - Distributed denial-of-service (DDoS) attacks employ multiple (dozens to millions) compromised computers to perform a coordinated and widely distributed DoS attack
  - Victims of DoS attacks

    - service-providers (interms of time, money, resources, goodwill)
    - Legitimate service-seekers (deprivedofavailabilityofserviceitself)
    - Zombie systems (Penultimate and previous layers of compromised systems in DDoS)



System Protection in Operating System

Introduction:
System protection in an operating system refers to the mechanisms implemented by the operating system to ensure the security and integrity of the system. System protection involves various techniques to prevent unauthorized access, misuse, or modification of the operating system and its resources.

There are several ways in which an operating system can provide system protection:
1. User authentication: The operating system requires users to authenticate themselves before accessing the system. Usernames and passwords are commonly used for this purpose

2. Access control: The operating system uses access control lists (ACLs) to determine which users or processes have permission to access specific resources or perform specific actions.

3. Encryption: The operating system can use encryption to protect sensitive data and prevent unauthorized access.

4. Firewall: A firewall is a software program that monitors and controls incoming and outgoing network traffic based on predefined security rules.

5. Antivirus software: Antivirus software is used to protect the system from viruses, malware, and other malicious software.

6. System updates and patches: The operating system must be kept up-to-date with the latest security patches and updates to prevent known vulnerabilities from being exploited.

By implementing these protection mechanisms, the operating system can prevent unauthorized access to the system, protect sensitive data, and ensure the overall security and integrity of the system.

Goals of Protection

- Obviously to prevent malicious misuse of the system by users or programs. See chapter 15 for a more thorough coverage of this goal.
- To ensure that each shared resource is used only in accordance with system *policies,* which may be set either by system designers or by system administrators.
- To ensure that errant programs cause the minimal amount of damage possible.
- Note that protection systems only provide the *mechanisms* for enforcing policies and ensuring reliable systems. It is up to administrators and users to implement those mechanisms effectively.

Principles of Protection

- The **principle of least privilege** dictates that programs, users, and systems be given just enough privileges to perform their tasks.

- This ensures that failures do the least amount of harm and allow the least of harm to be done.
- For example, if a program needs special privileges to perform a task, it is better to make it a SGID program with group ownership of "network" or "backup" or some other pseudo group, rather than SUID with root ownership. This limits the amount of damage that can occur if something goes wrong.
- Typically each user is given their own account, and has only enough privilege to modify their own files.
- The root account should not be used for normal day to day activities - The System Administrator should also have an ordinary account, and reserve use of the root account for only those tasks which need the root privileges

Domain of Protection

- A computer can be viewed as a collection of *processes* and *objects* (both HW & SW).
- The **need to know principle** states that a process should only have access to those objects it needs to accomplish its task, and furthermore only in the modes for which it needs access and only during the time frame when it needs access.
- The modes available for a particular object may depend upon its type.

Domain Structure

- A **protection domain** specifies the resources that a process may access.
- Each domain defines a set of objects and the types of operations that may be invoked on each object.
- An **access right** is the ability to execute an operation on an object.
- A domain is defined as a set of < object, { access right set } > pairs, as shown below. Note that some domains may be disjoint while others overlap.

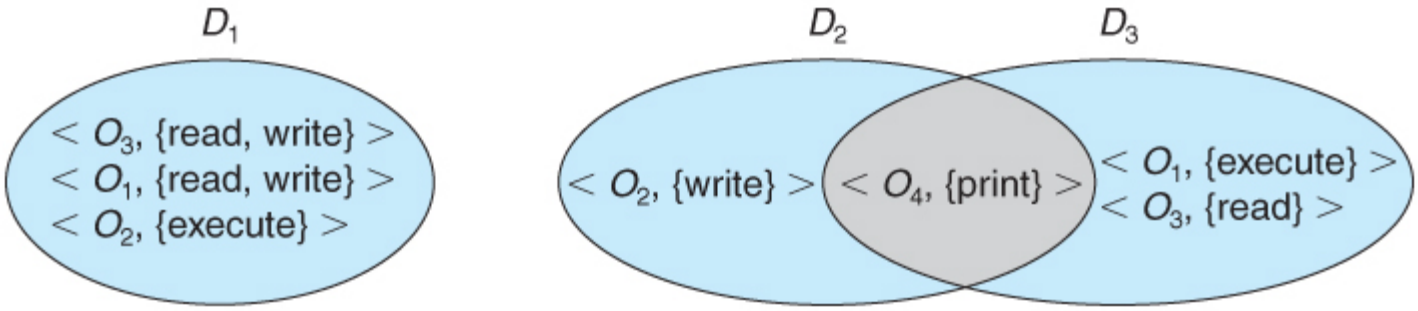


Figure 14.1 - System with three protection domains.

- The association between a process and a domain may be *static* or *dynamic.*
    - If the association is static, then the need-to-know principle requires a way of changing the contents of the domain dynamically.
    - If the association is dynamic, then there needs to be a mechanism for **domain switching.**

- Domains may be realized in different fashions - as users, or as processes, or as procedures. E.g. if each user corresponds to a domain, then that domain defines the access of that user, and changing domains involves changing user ID.

Access matrix in Operating System

Access Control Matrix (ACM) is a security model of protection state in computer system. It is represented as a matrix. Access matrix is used to define the rights of each process executing in the domain with respect to each object. The rows of matrix represent domains and columns represent objects. Each cell of matrix represents set of access rights which are given to the processes of domain means each entry(i, j) defines the set of operations that a process executing in domain Di can invoke on object Oj.

Different types of rights:

There are different types of rights the files can have. The most common ones are:

1. Read- This is a right given to a process in a domain, which allows it to read the file.
2. Write- Process in domain can write into the file.
3. Execute- Process in domain can execute the file.
4. Print- Process in domain only has access to printer.

Sometimes, domains can have more than one right, i.e. combination of rights mentioned above. Let us now understand how an access matrix works from the example given below.

	F1	**F2**	**F3**	**Printer**
D1	read		read	
D2				print
D3		read	execute	
D4	read write		read write	

Observations of above matrix:

- There are **four domains** and **four objects**– three files(F1, F2, F3) and one printer.
- A process executing in D1 can read files F1 and F3.
- A process executing in domain D4 has same rights as D1 but it can also write on files.
- Printer can be accessed by only one process executing in domain D2.
- A process executing in domain D3 has the right to read file F2 and execute file F3.

Mechanism of access matrix:

The mechanism of access matrix consists of many policies and semantic properties. Specifically, we must ensure that a process executing in domain Di can access only those objects that are specified in row i. Policies of access matrix concerning protection involve which rights should be included in the **(i, j)th entry**. We must also decide the domain in which each process executes. This policy is usually decided by the operating system. The users decide the contents of the access-matrix entries. Association between the domain and processes can be either static or dynamic. Access matrix provides a mechanism for defining the control for this association between domain and processes.

Switch operation: When we switch a process from one domain to another, we execute a switch

operation on an object(the domain). We can control domain switching by including domains among the objects of the access matrix. Processes should be able to switch from one domain (Di) to another domain (Dj) if and only if a switch right is given to access(i, j). This is explained using an example below:

	F1	**F2**	**F3**	**Printer**	**D1**	**D2**	**D3**	**D4**
D1	read		read			switch		
D2				print			switch	switch
D3		read	execute					
D4	read write		read write		switch			

According to the above matrix, a process executing in domain D2 can switch to domain D3 and D4. A process executing in domain D4 can switch to domain D1 and process executing in domain D1 can switch to domain D2.

Access-Lists (ACL)

Access-list (ACL) is a set of rules defined for controlling network traffic and reducing network attacks. ACLs are used to filter traffic based on the set of rules defined for the incoming or outgoing of the network.

ACL features –

1. The set of rules defined are matched serial wise i.e matching starts with the first line, then 2nd, then 3rd, and so on.
2. The packets are matched only until it matches the rule. Once a rule is matched then no further comparison takes place and that rule will be performed.
3. There is an implicit denial at the end of every ACL, i.e., if no condition or rule matches then the packet will be discarded.

Advantages of ACL –

- Improve network performance.
- Provides security as the administrator can configure the access list according to the needs and deny the unwanted packets from entering the network.
- Provides control over the traffic as it can permit or deny according to the need of the network.